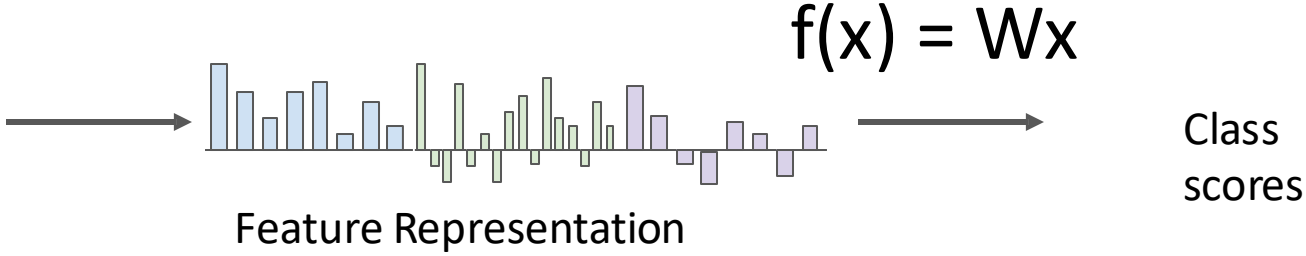
Image Classification with CNNs

Pixel space

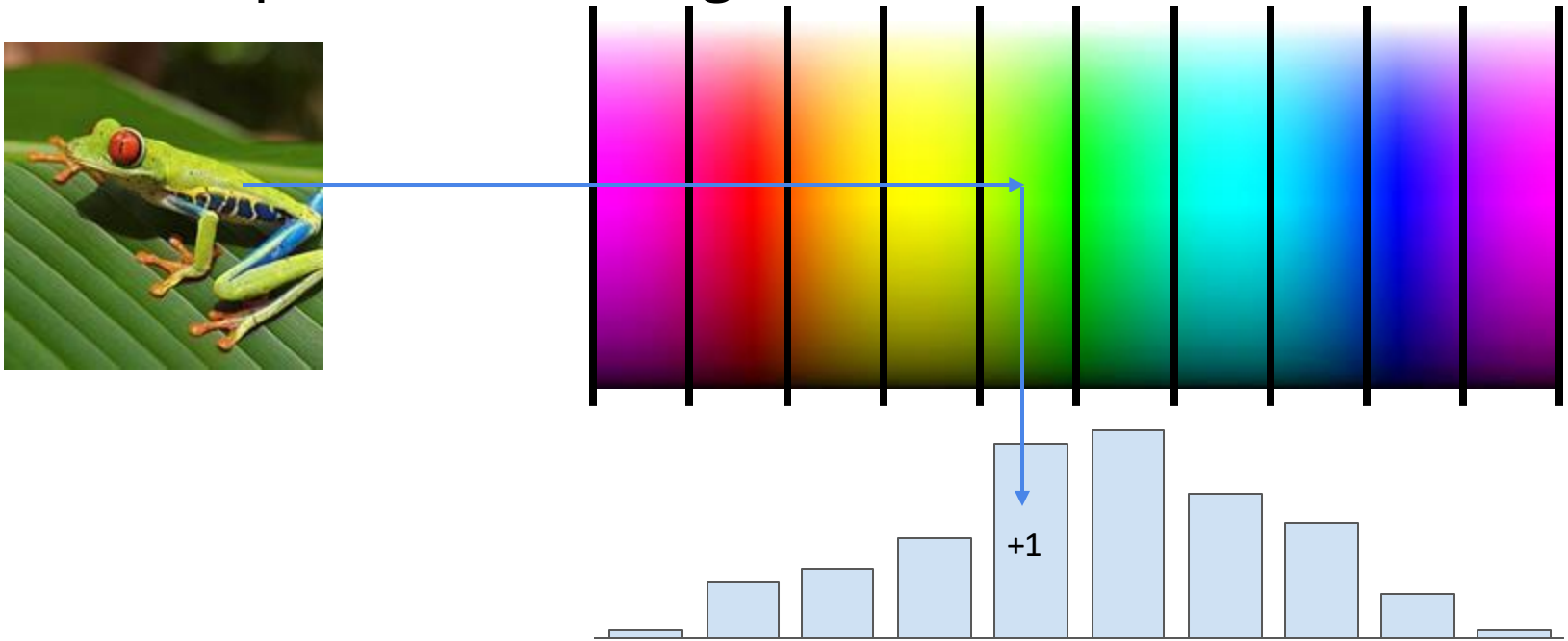
Class scores

$$f(x) = Wx$$

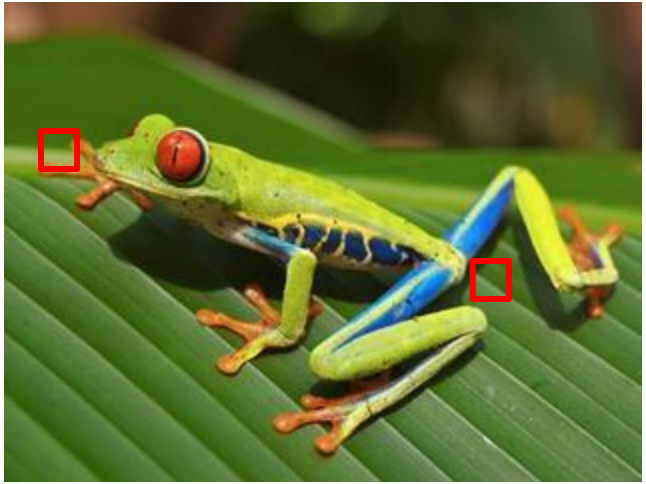
Image features



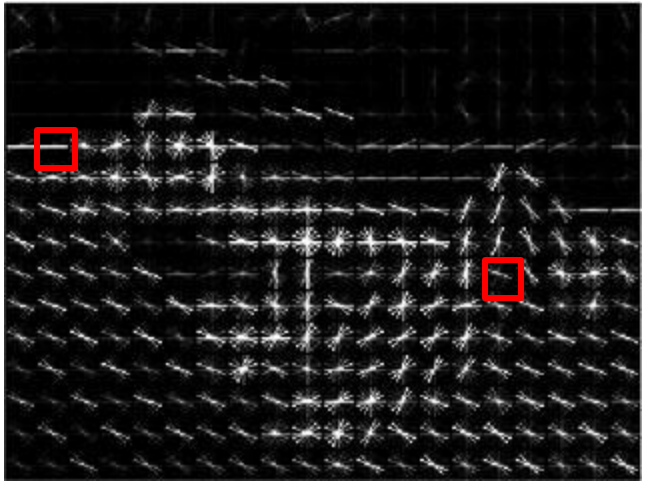
Example: Color Histogram



Example: Histogram of Oriented Gradients (HoG)



Divide image into 8x8 pixel regions
Within each region quantize edge direction into 9 bins



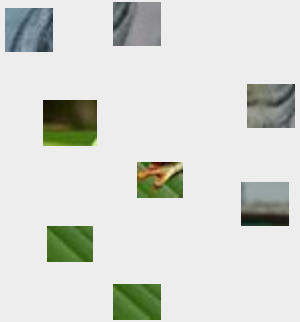
Example: 320x240 image gets divided into 40x30 bins; in each bin there are 9 numbers so feature vector has 30*40*9 = 10,800 numbers

Lowe, “Object recognition from local scale-invariant features”, ICCV 1999
Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

Example: Bag of Words

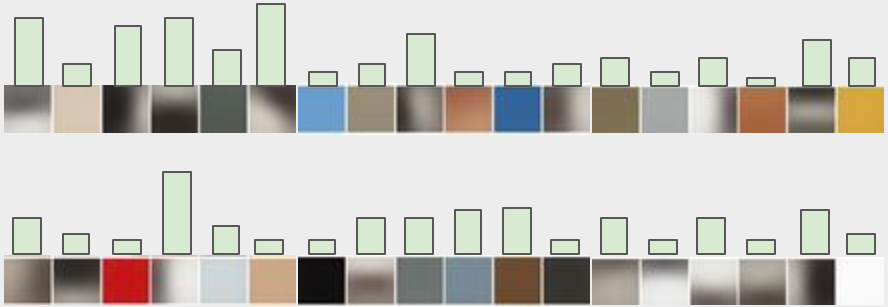
Step 1: Build codebook

Extract random patches



Cluster patches to form "codebook" of "visual words"

Step 2: Encode images



Fei-Fei and Perona, "A bayesian hierarchical model for learning natural scene categories", CVPR 2005

Image Features

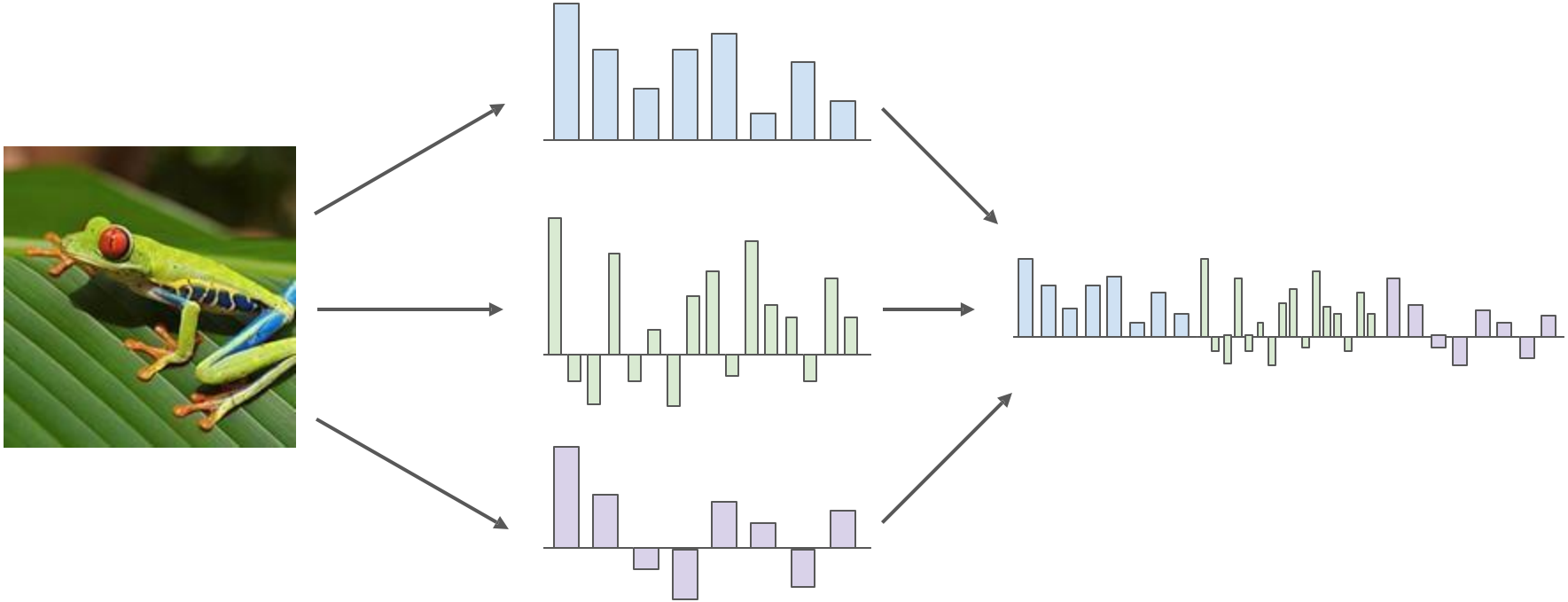


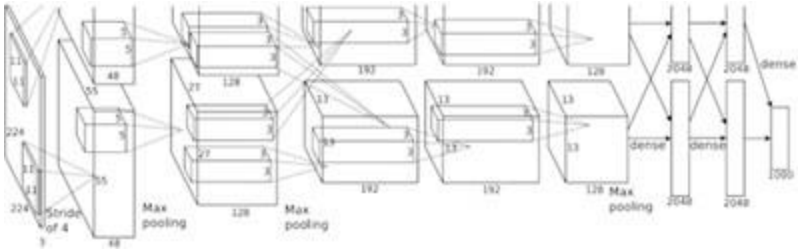
Image features vs. ConvNets

Feature Extraction

f

10 numbers giving scores for classes

training



Krizhevsky, Sutskever, and Hinton, "Imagenet classification with deep convolutional neural networks", NIPS 2012. Figure copyright Krizhevsky, Sutskever, and Hinton, 2012. Reproduced with permission.

10 numbers giving scores for classes

training

Last Time: Neural Networks

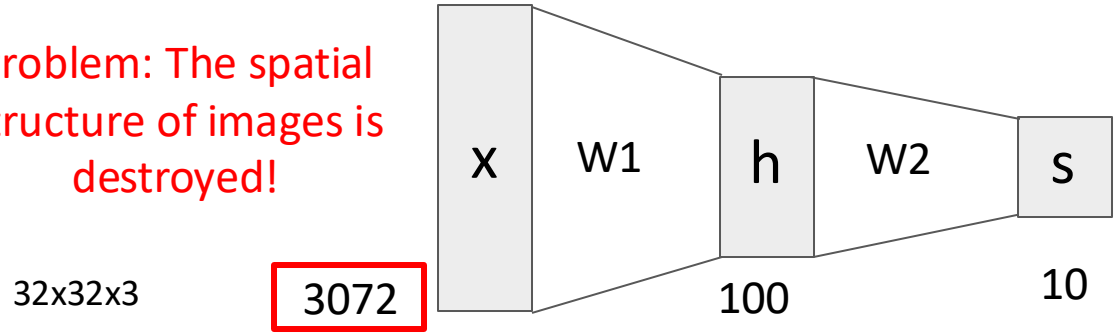
Linear score function:

$$f = Wx$$

2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

Problem: The spatial structure of images is destroyed!



Next: Convolutional Neural Networks

Convolution and **pooling** operators extract features while respecting 2D image structure

Fully-Connected layers form an MLP at the end to predict scores

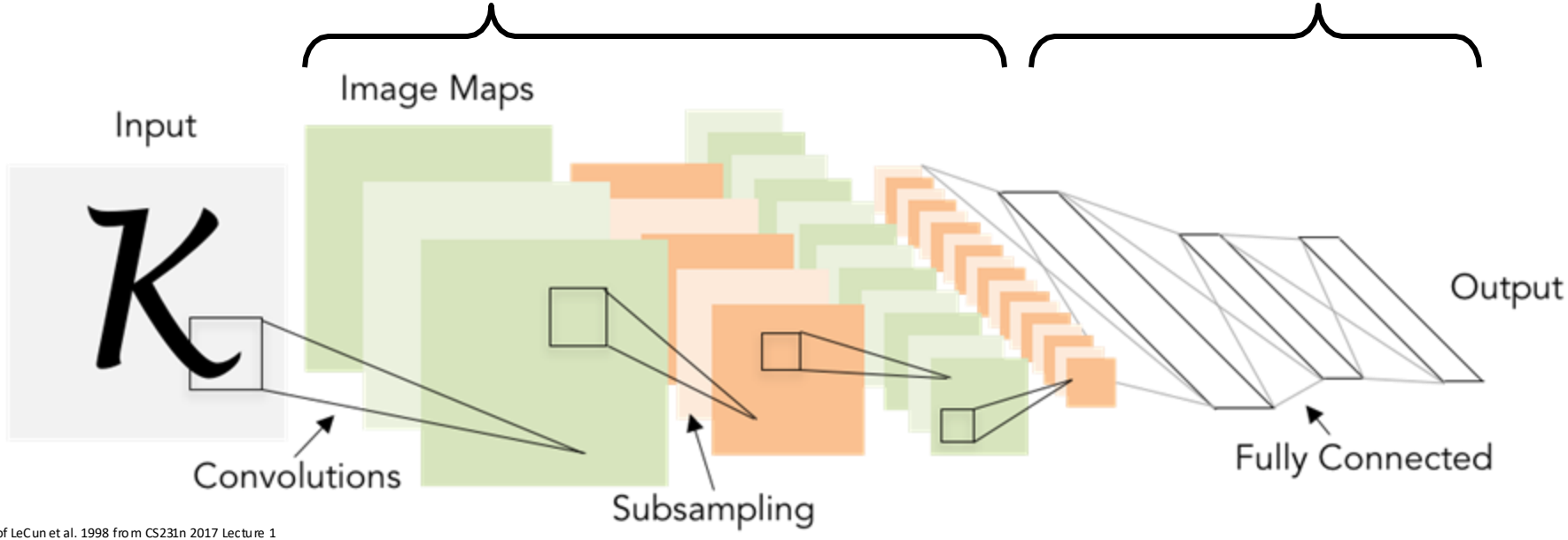


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

Next: Convolutional Neural Networks

Trained end-to-end with backprop + gradient descent

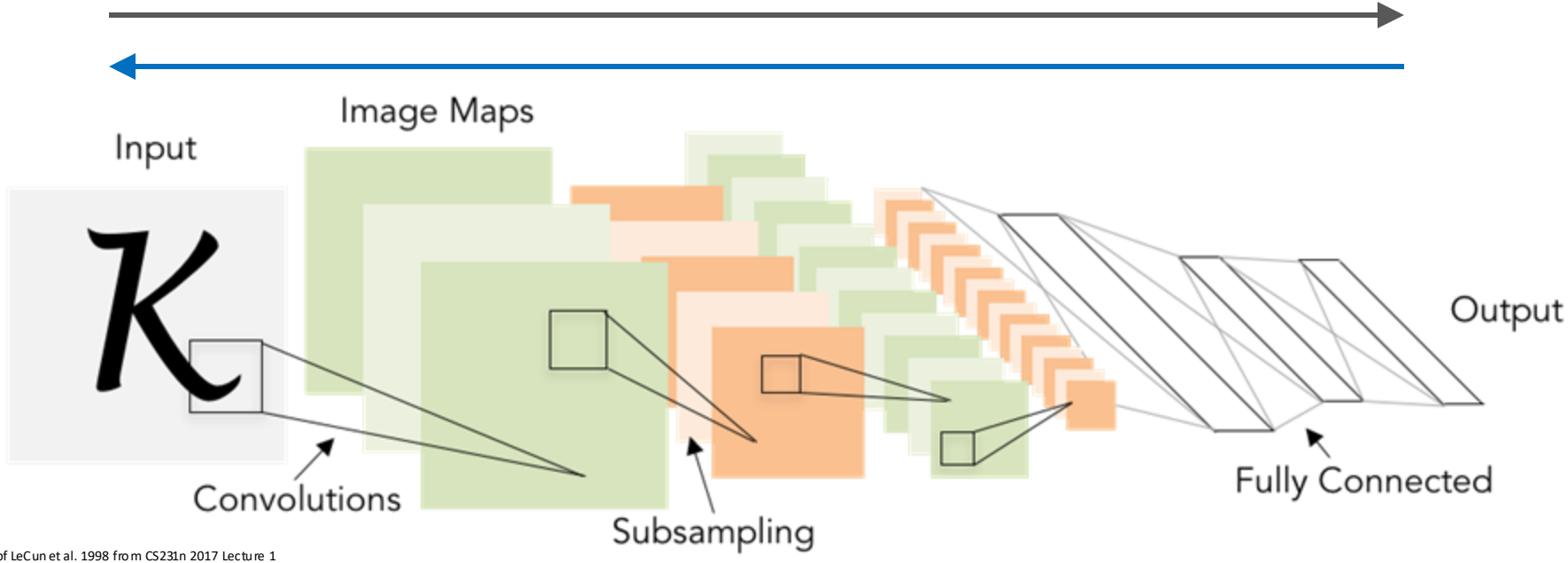
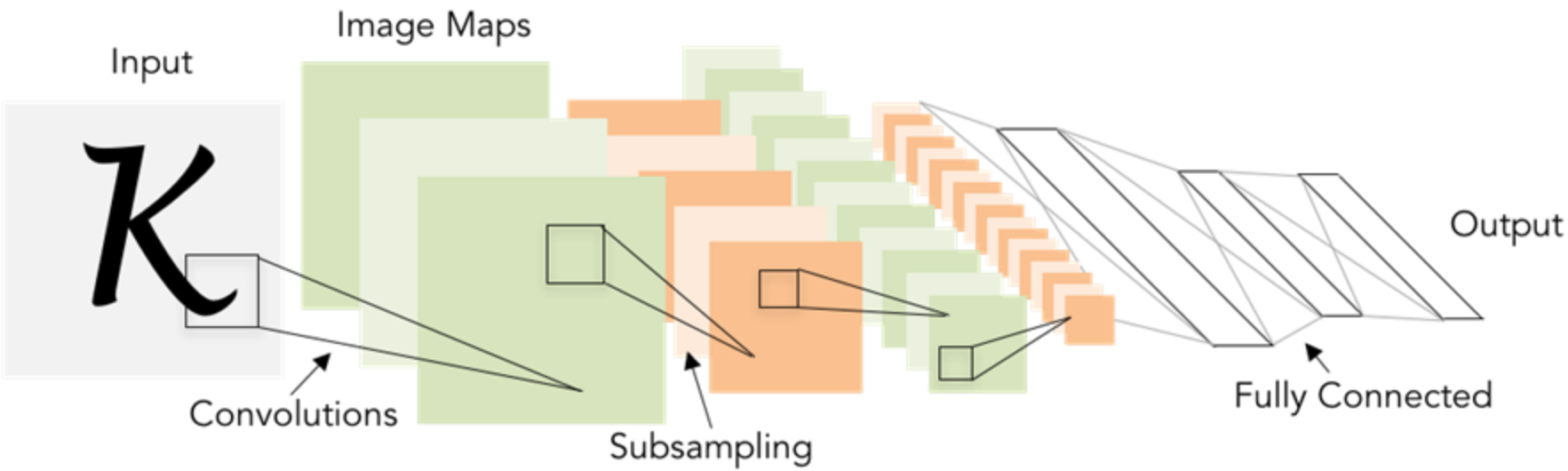


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

A bit of history:

Gradient-based learning applied to document recognition
[LeCun, Bottou, Bengio, Haffner 1998]



A bit of history:

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky, Sutskever, Hinton, 2012]

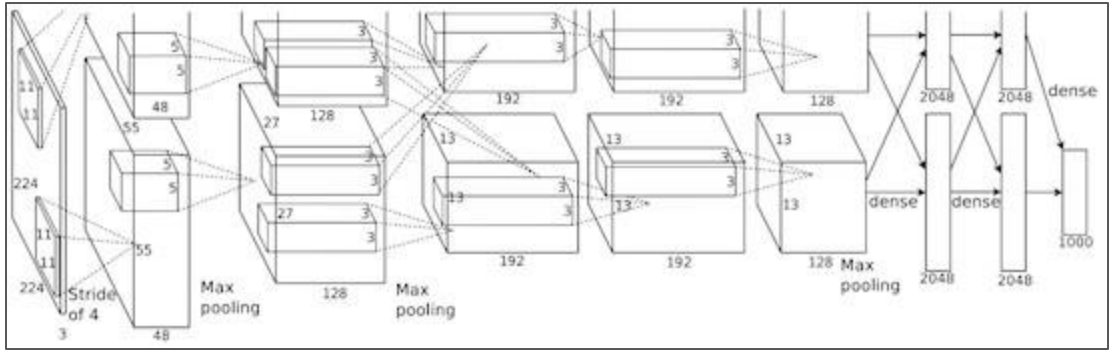


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

"AlexNet"

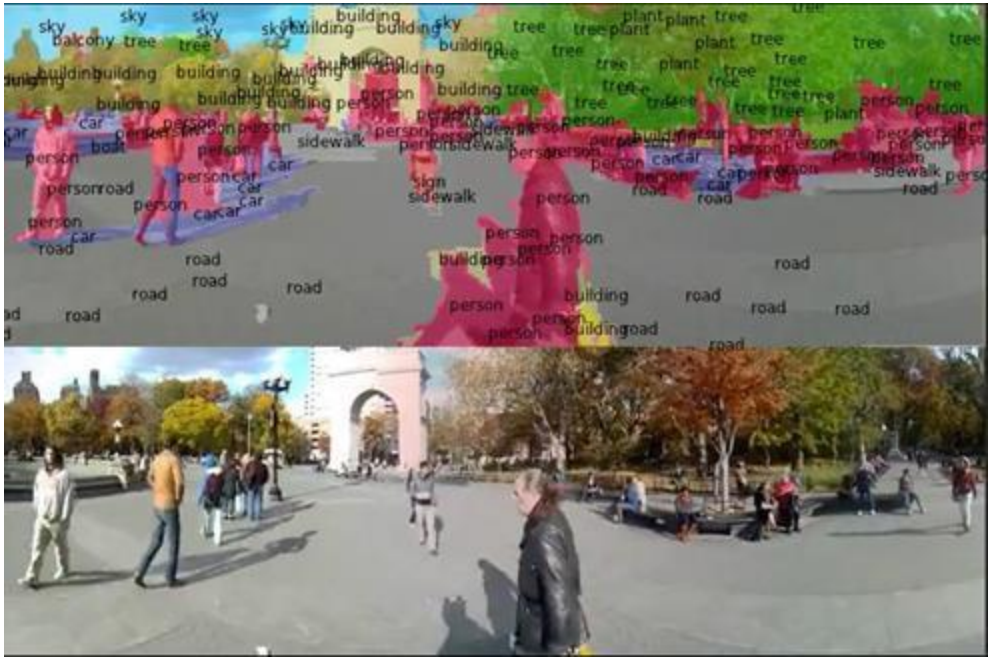
~2012 – 2020: ConvNets dominate all vision tasks

Detection

Figures copyright Shaoqing Ren, Kaiming He, Ross Girschick, Jian Sun, 2015. Reproduced with permission.

[Faster R-CNN: Ren, He, Girshick, Sun 2015]

Detection



Figures copyright Clement Farabet, 2012. Reproduced with permission.

[Farabet et al., 2012]

~2012 – 2020: ConvNets dominate all vision tasks

Image Captioning

A white teddy bear sitting in the grass

A man in a baseball uniform throwing a ball

A woman is holding a cat in her hand

A man riding a wave on top of a surfboard

A cat sitting on a suitcase on the floor

A woman standing on a beach holding a surfboard

All images are CC0 Public domain:
https://pixabay.com/en/luggage-antique-cat-1643010/
https://pixabay.com/en/teddy-plush-bears-cute-teddy-bear-1623436/
https://pixabay.com/en/surf-wave-summer-sport-litoral-1668716/
https://pixabay.com/en/woman-female-model-portrait-adult-983967/
https://pixabay.com/en/handstand-lake-meditation-496008/
https://pixabay.com/en/baseball-player-shortstop-infield-1045263/

Captions generated by Justin Johnson using Neuraltalk2

[Vinyals et al., 2015]
[Karpathy and Fei-Fei, 2015]

~2012 – 2020: ConvNets dominate all vision tasks

Text-to-Image Generation

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

A zombie in the style of Picasso

An image of a half mouse half octopus

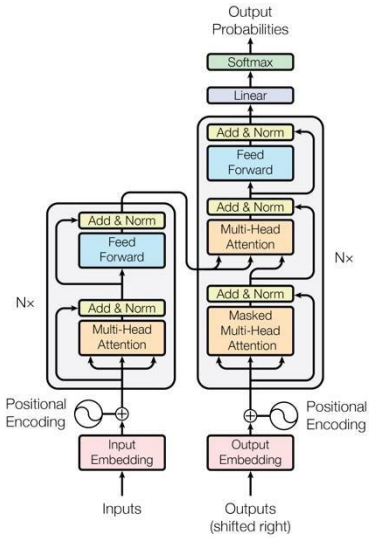
A painting of a squirrel eating a burger

A watercolor painting of a chair that looks like an octopus

A shirt with the inscription: "I love generative models!"

2021 - Present: Transformers have taken over

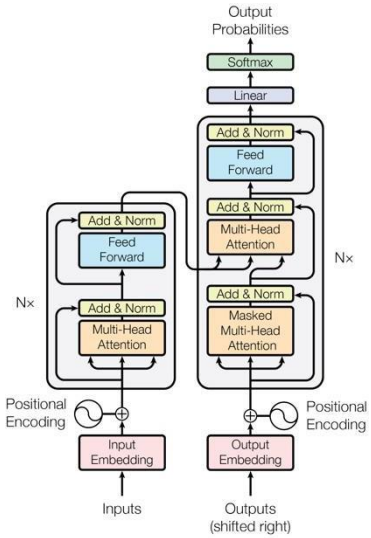
2017: Transformers
for language tasks



Vaswani et al, “Attention is all you need”, NeurIPS 2017

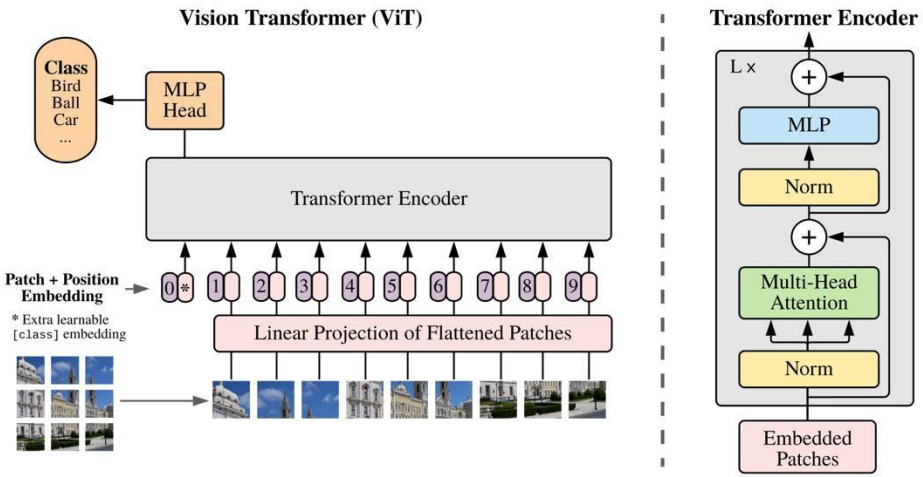
2021 - Present: Transformers have taken over

2017: Transformers for language tasks



Vaswani et al, “Attention is all you need”, NeurIPS 2017

2021: Transformers for vision tasks



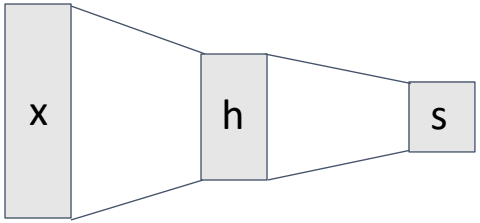
Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Convolutional Neural Networks

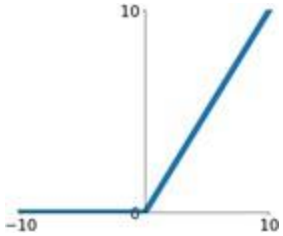
Convolutional Networks

We have already seen these

Fully-Connected Layer

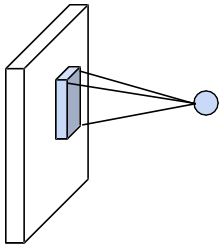


Activation Function

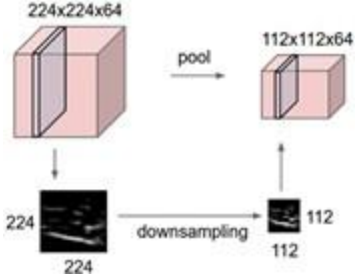


Today: Image-specific operators

Convolution Layer

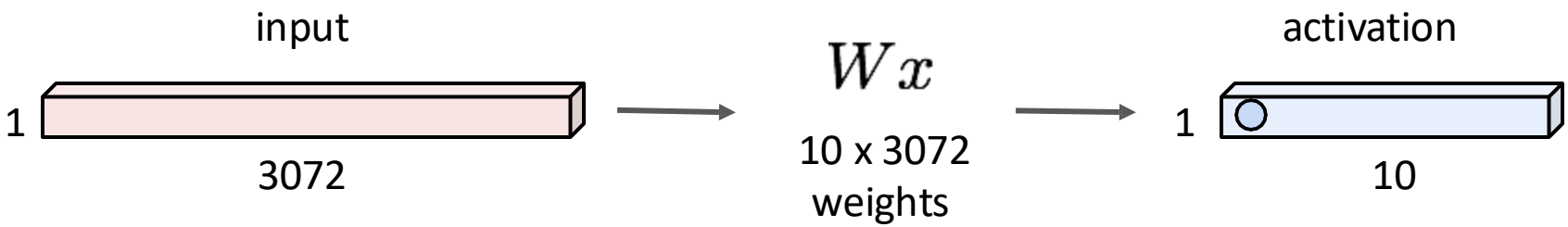


Pooling Layer



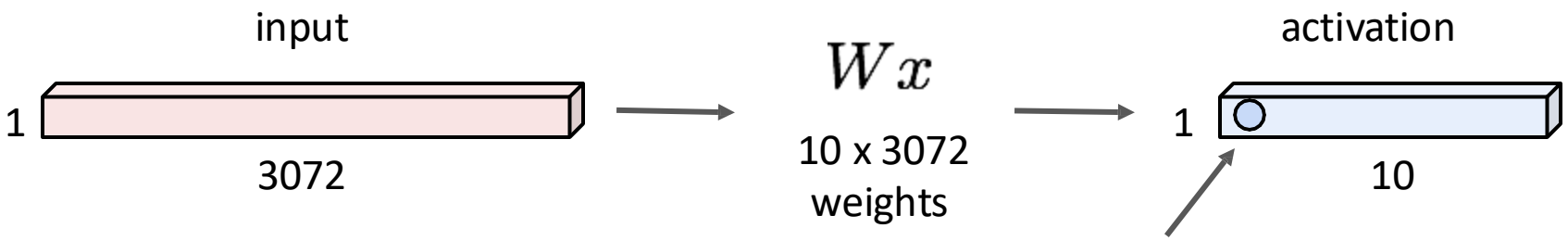
Recap: Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



Fully Connected Layer

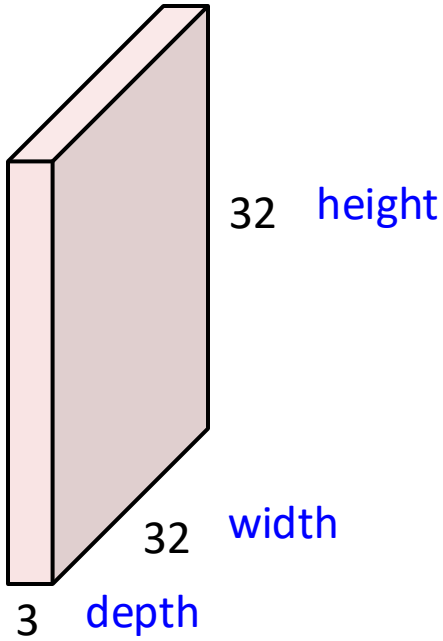
32x32x3 image -> stretch to 3072 x 1



1 number:
the result of taking a dot product between a row of W and the input (a 3072-dimensional dot product)

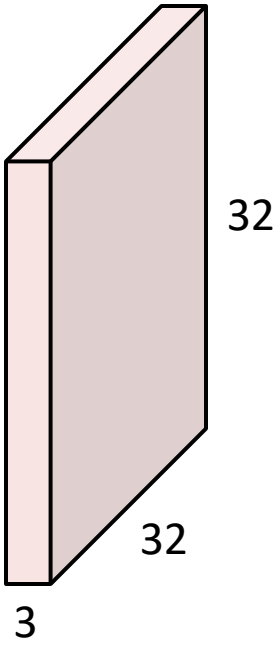
Convolution Layer

32x32x3 image -> preserve spatial structure

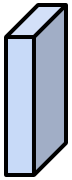


Convolution Layer

32x32x3 image



5x5x3 filter

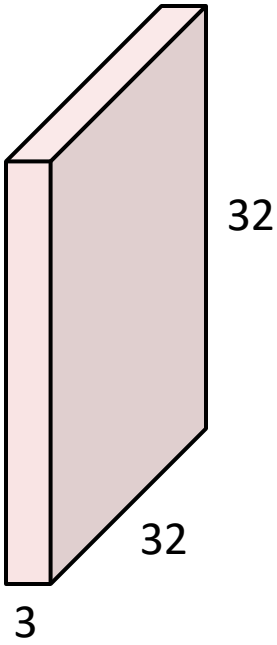


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

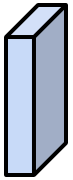
Convolution Layer

Filters always extend the full depth of the input volume

32x32x3 image

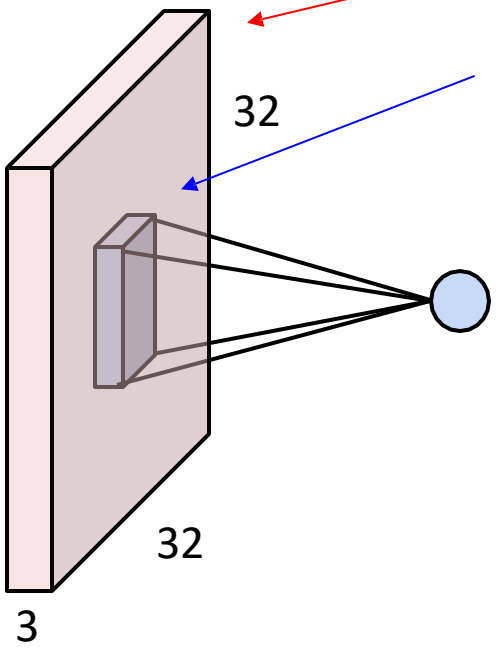


5x5x3 filter



Convolve the filter with the image i.e. “slide over the image spatially, computing dot products”

Convolution Layer



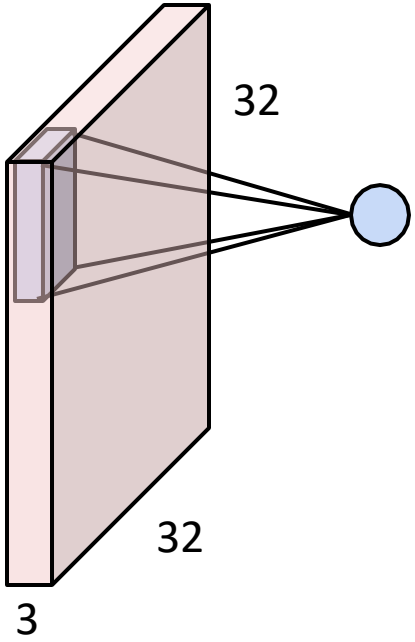
32x32x3 image

5x5x3 filter w

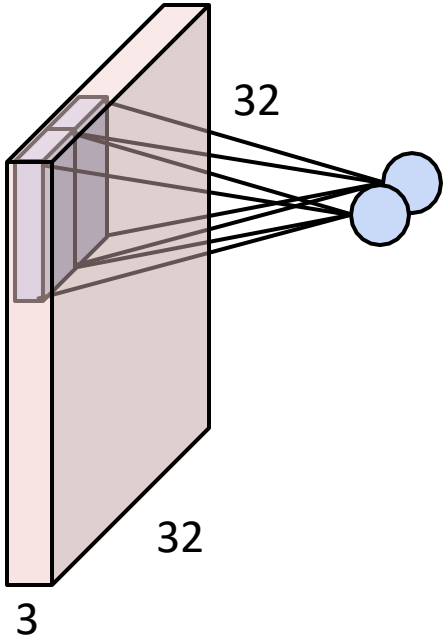
1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

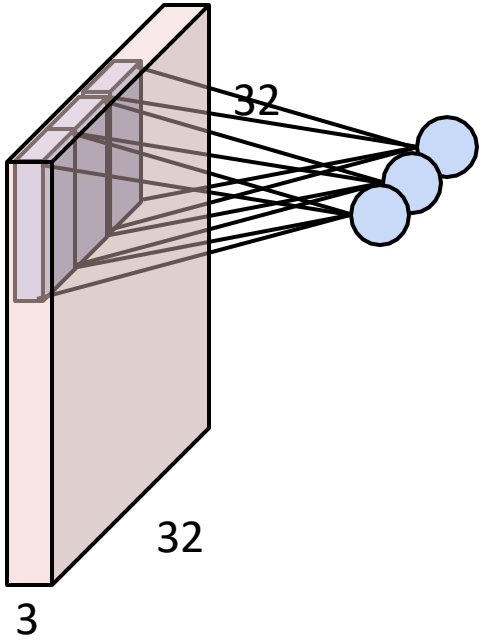
Convolution Layer



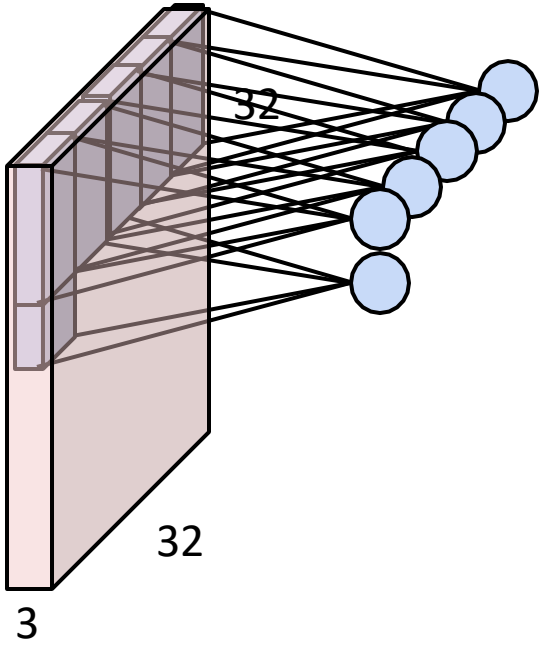
Convolution Layer



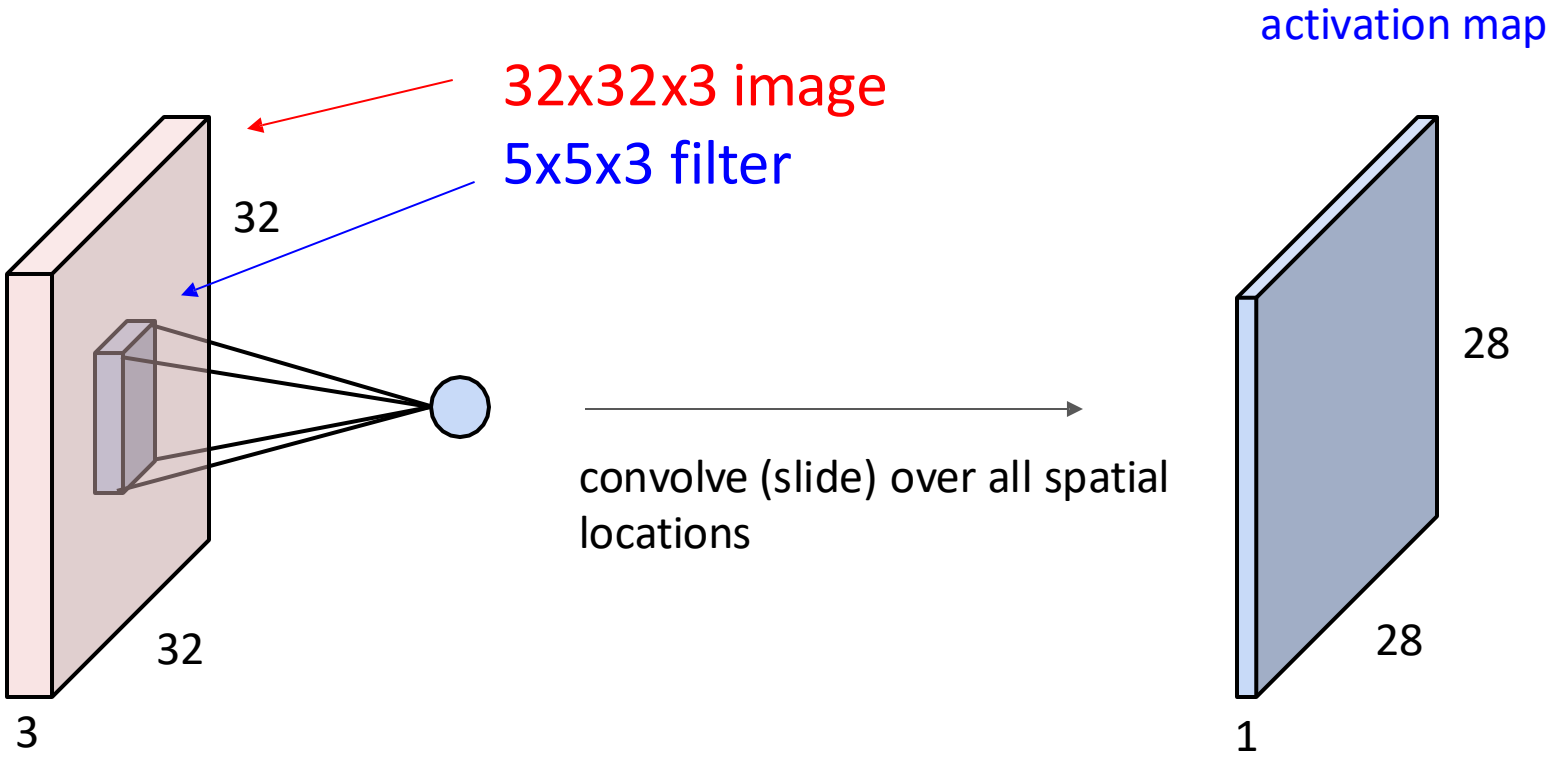
Convolution Layer



Convolution Layer

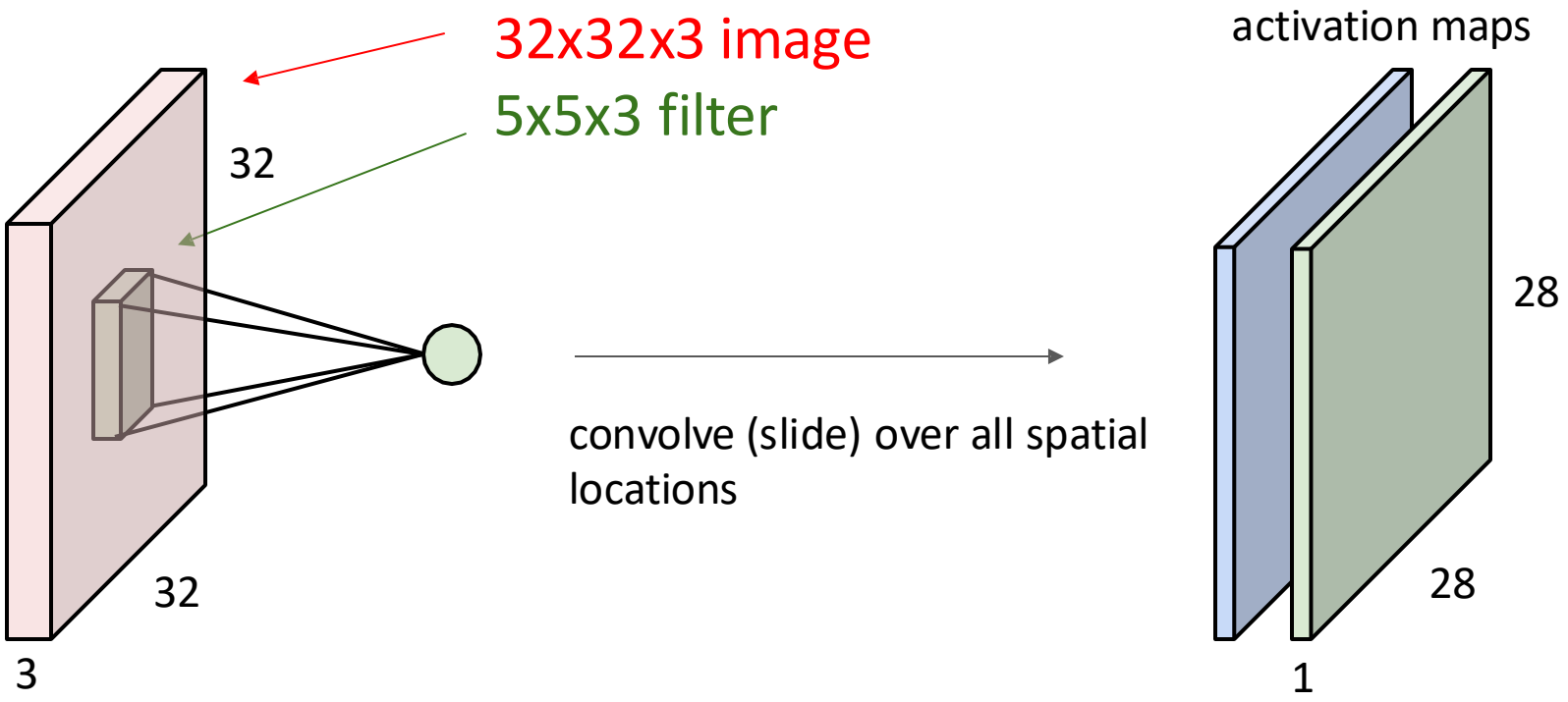


Convolution Layer

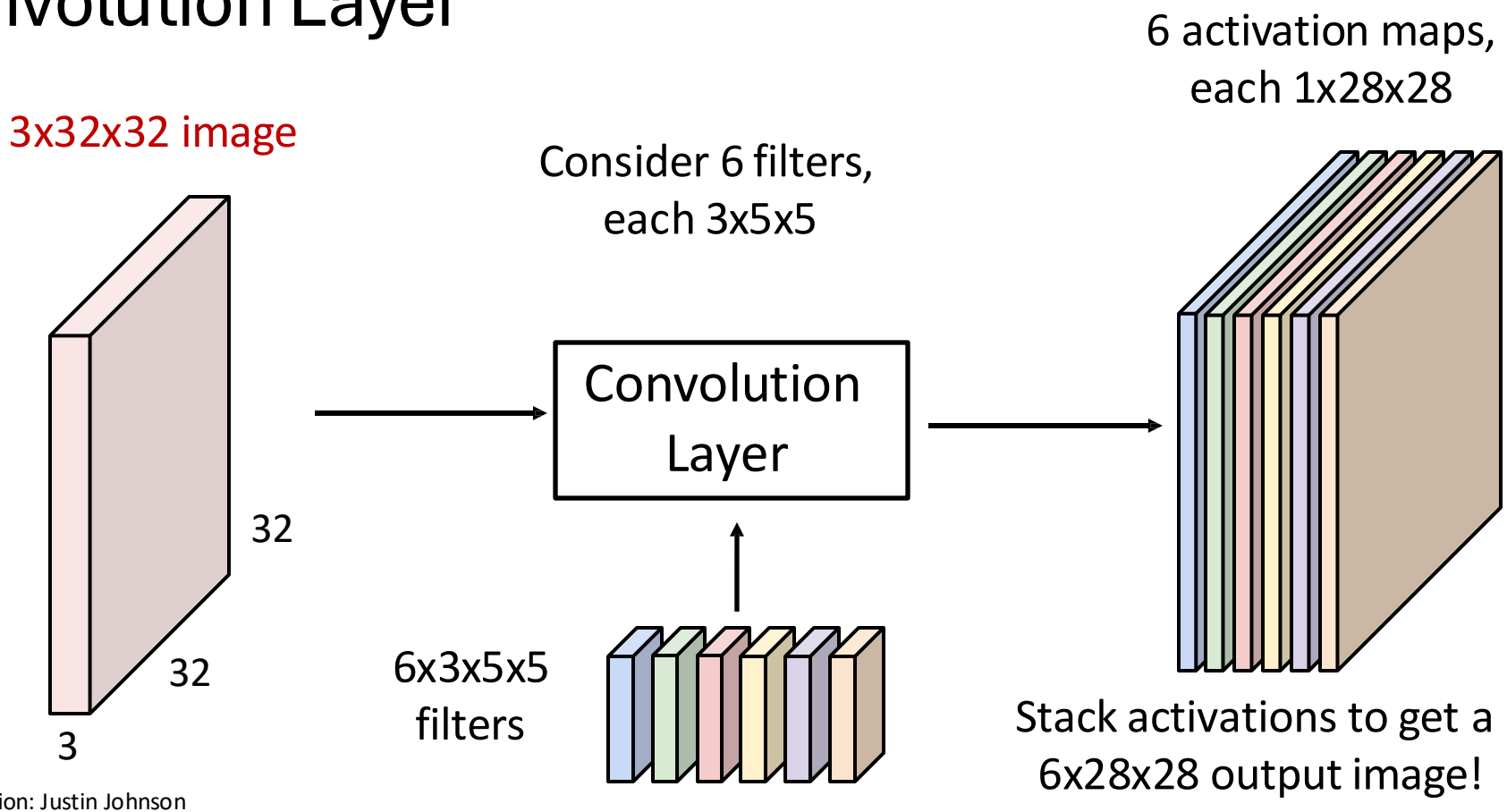


Convolution Layer

consider a second, green filter

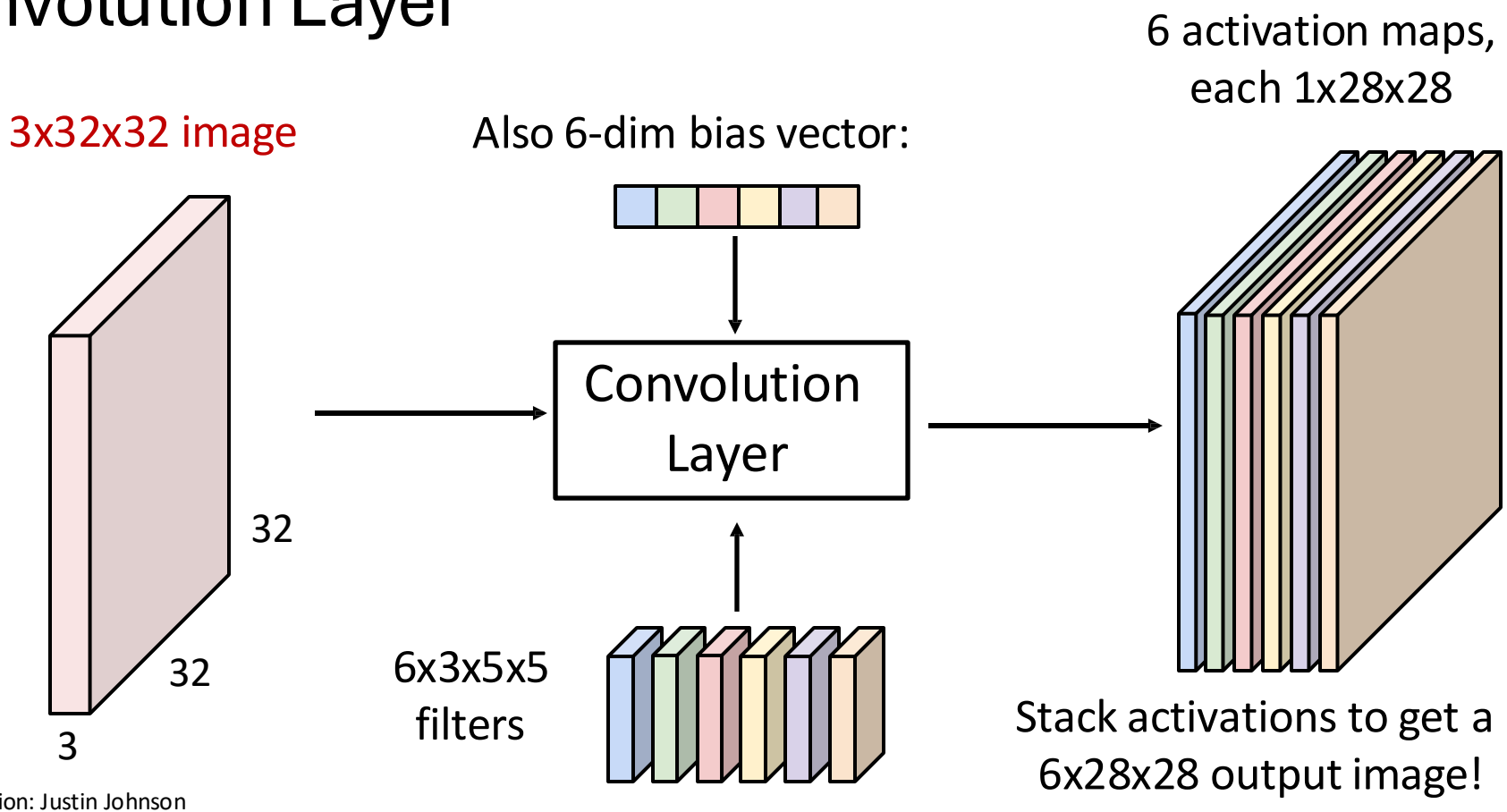


Convolution Layer



Slide inspiration: Justin Johnson

Convolution Layer



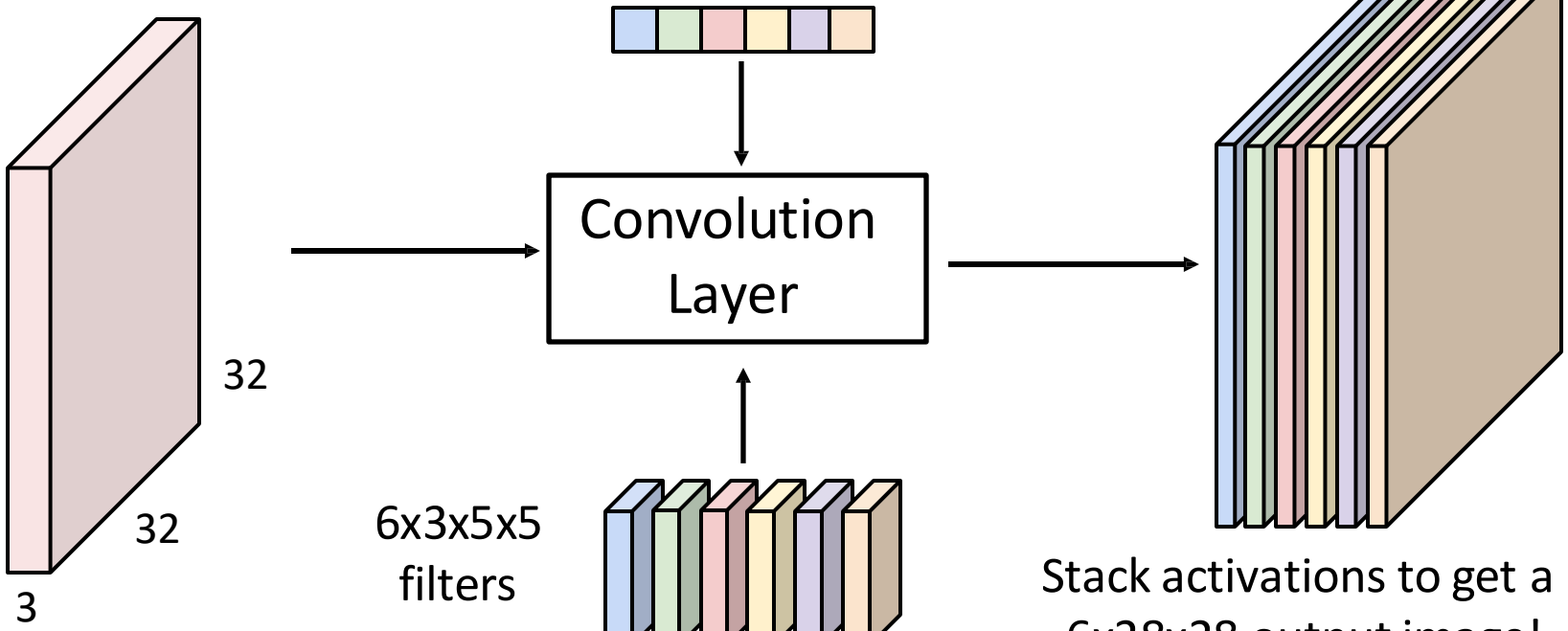
Slide inspiration: Justin Johnson

Convolution Layer

28x28 grid, at each point a 6-dim vector

3x32x32 image

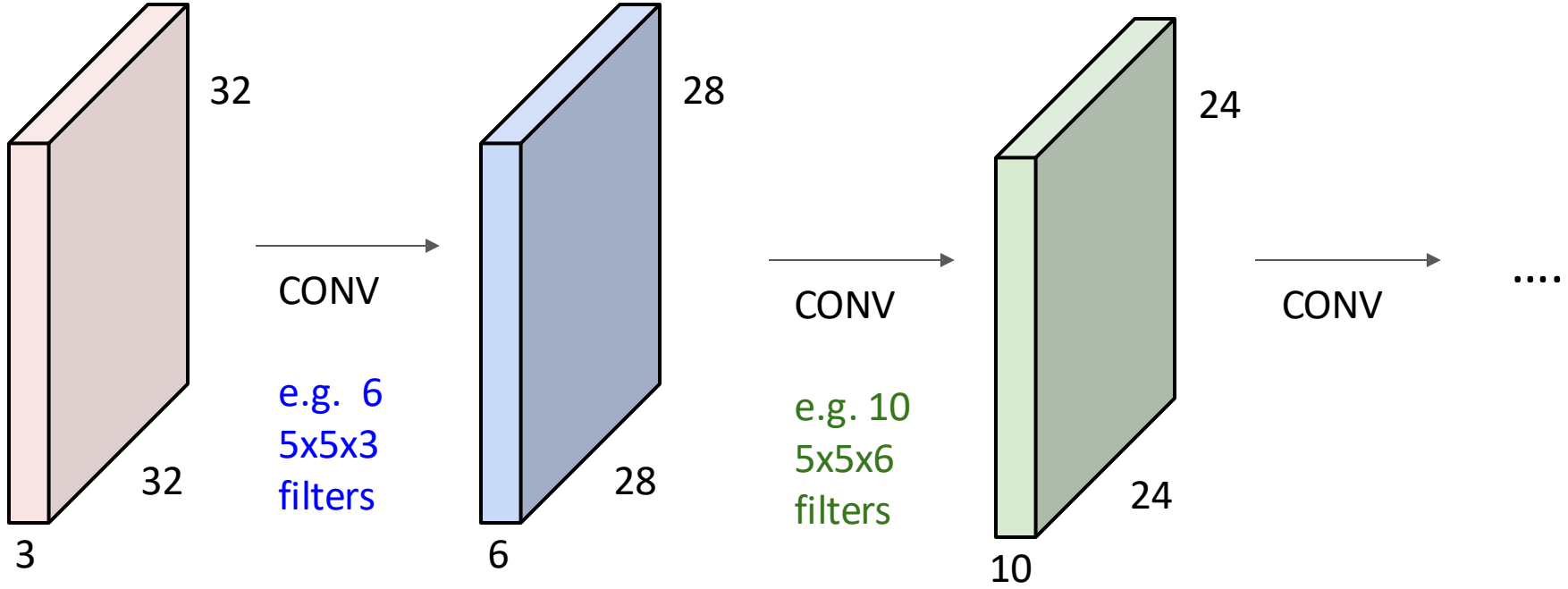
Also 6-dim bias vector:



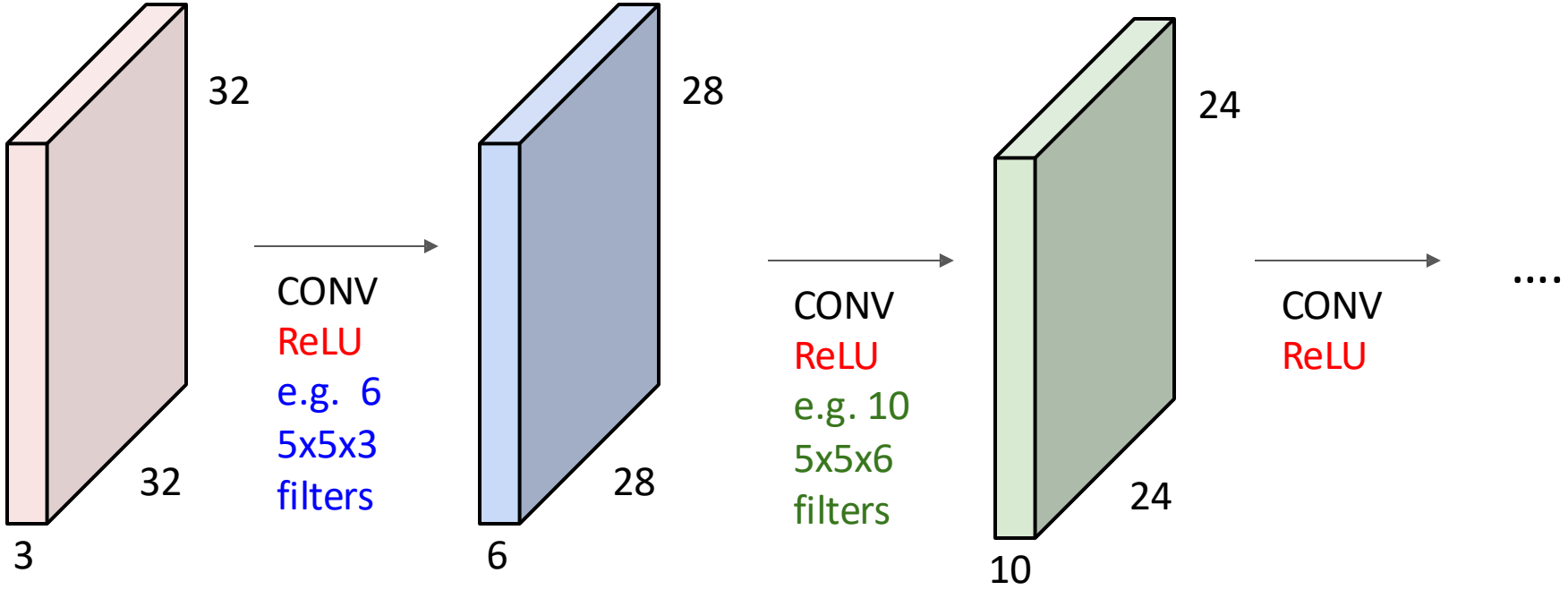
Stack activations to get a 6x28x28 output image!

Slide inspiration: Justin Johnson

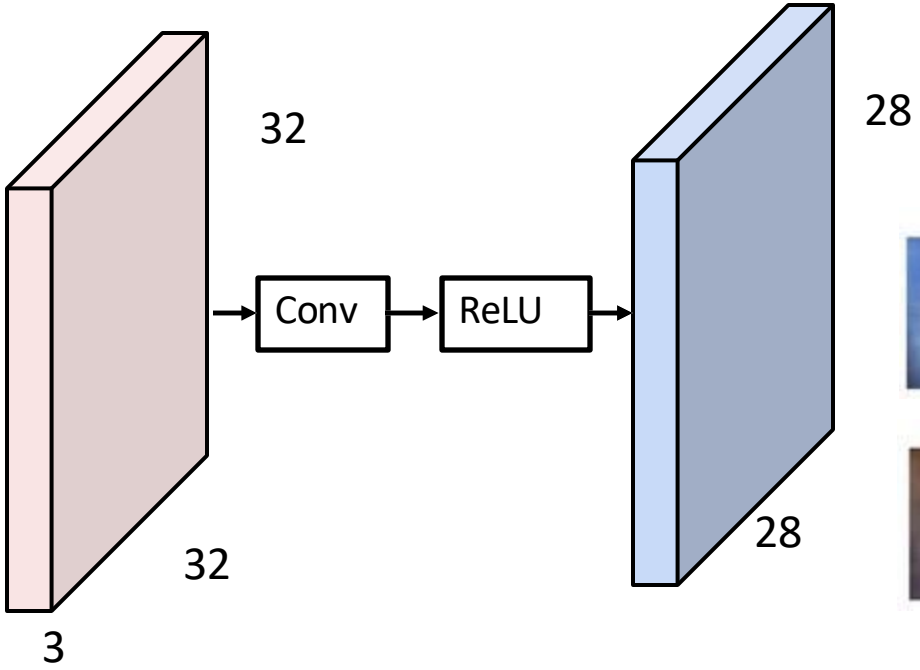
A ConvNet is a neural network with Conv layers



A ConvNet is a neural network with Conv layers with activation functions!

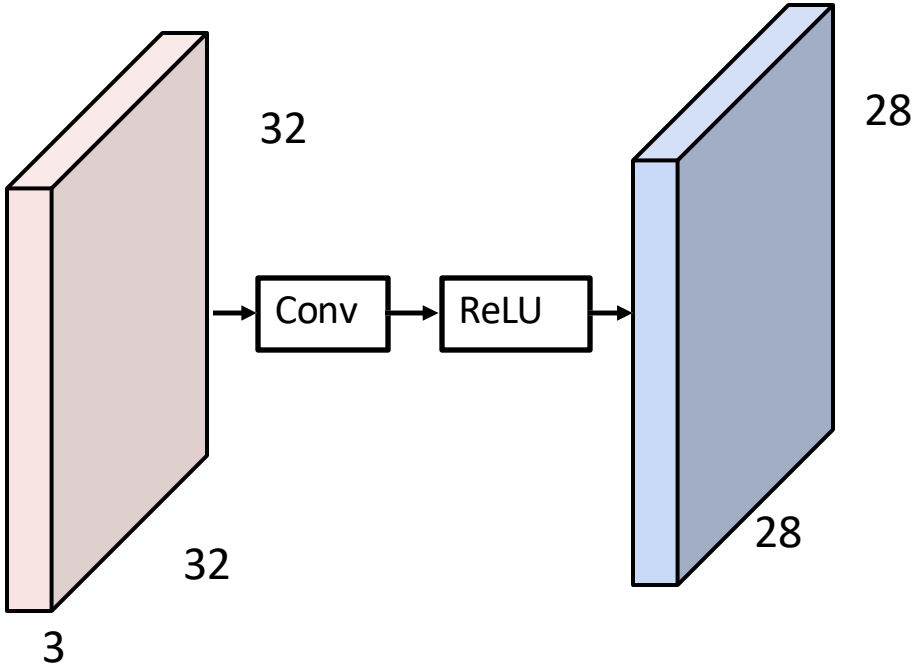


What do Conv filters learn?

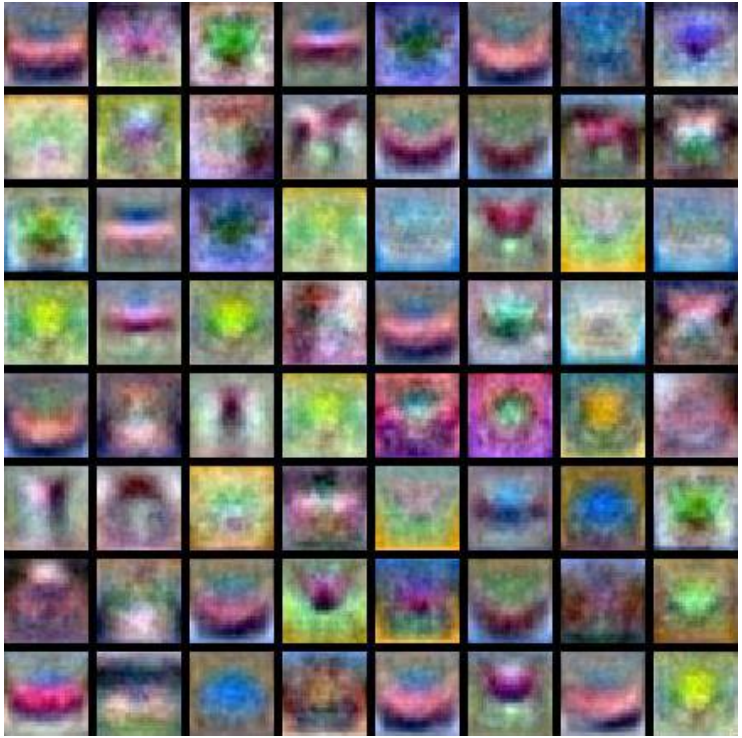


Linear classifier: One template per class

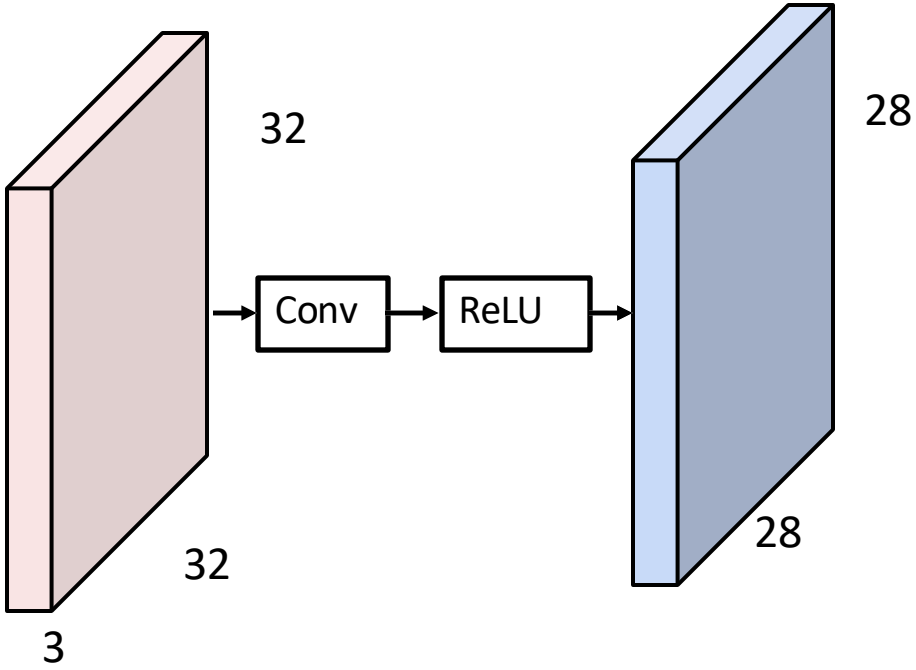
What do Conv filters learn?



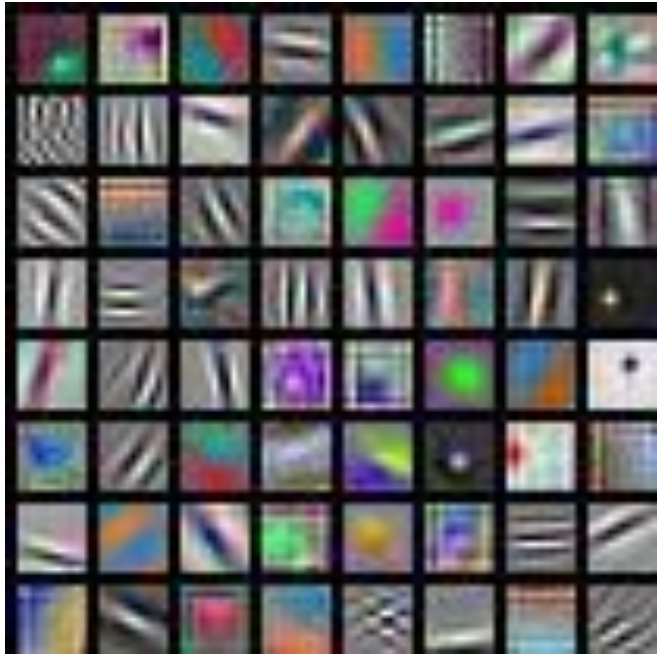
MLP: Bank of whole-image templates



What do Conv filters learn?

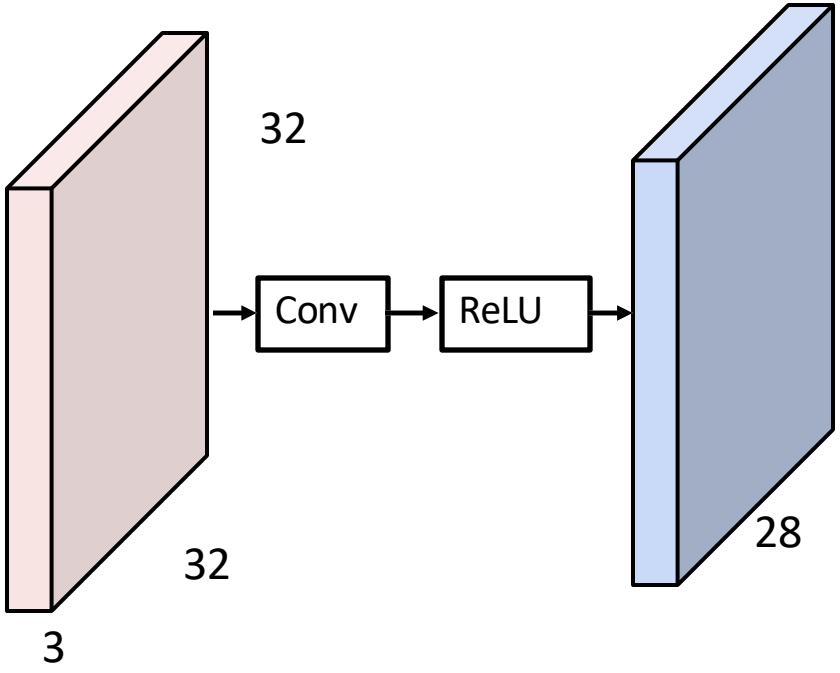


First-layer conv filters: local image templates
(Often learns oriented edges, opposing colors)

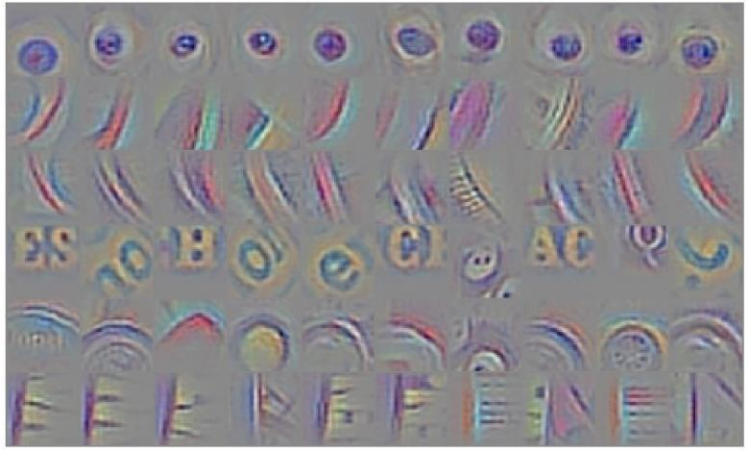


AlexNet: 64 filters, each 3x11x11

What do Conv filters learn?



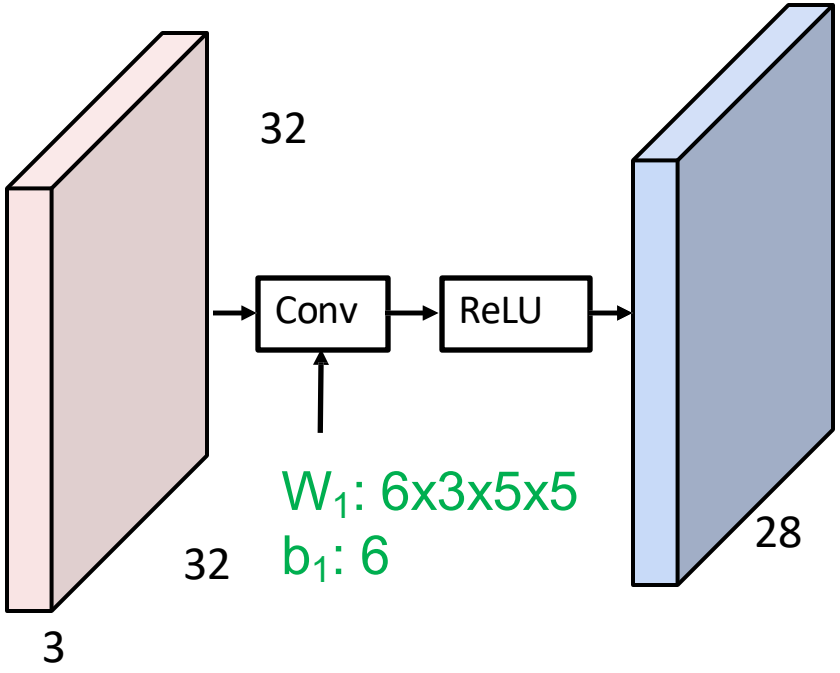
Deeper conv layers: Harder to visualize
Tend to learn larger structures e.g. eyes, letters



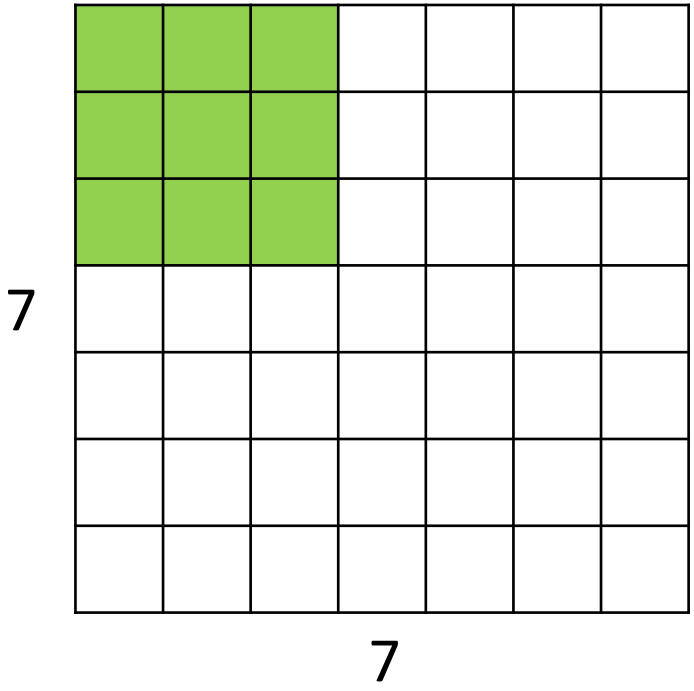
6th layer conv layer from an ImageNet model

Visualization from [Springenberg et al, ICLR 2015]

Convolution: Spatial Dimensions



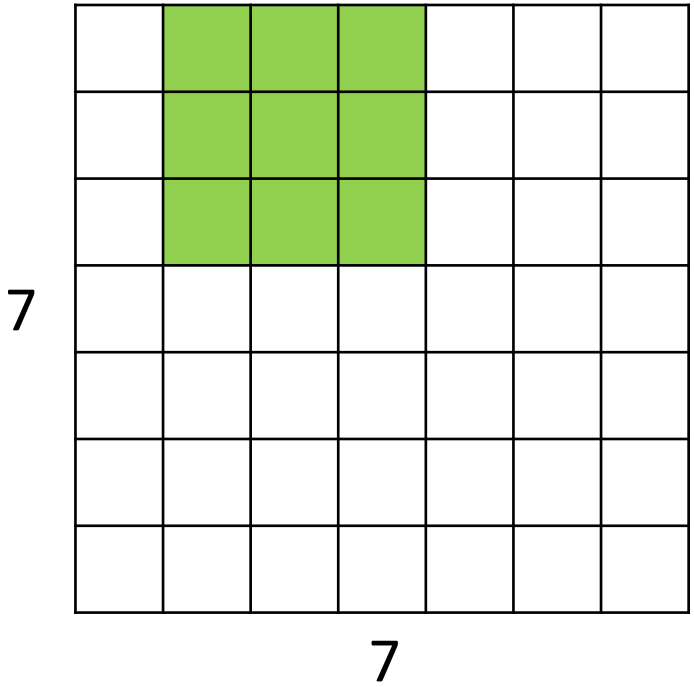
Convolution: Spatial Dimensions



Input: 7x7

Filter: 3x3

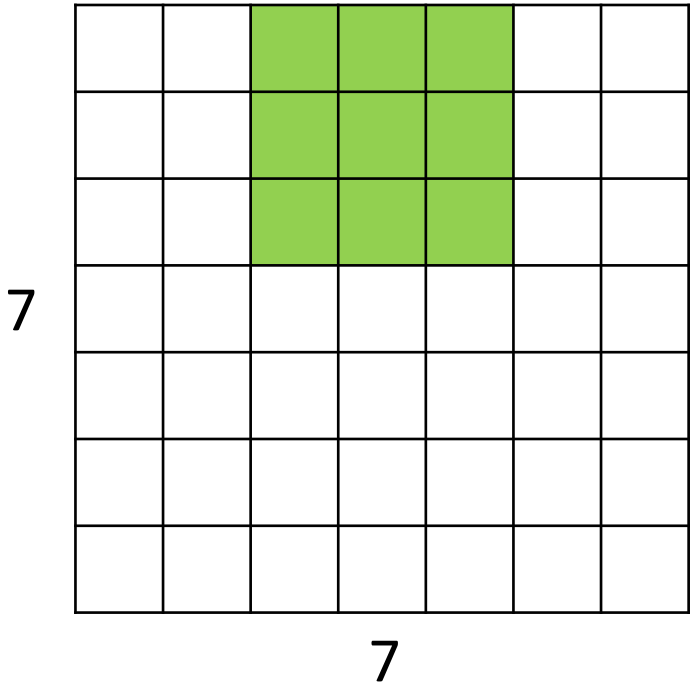
Convolution: Spatial Dimensions



Input: 7x7

Filter: 3x3

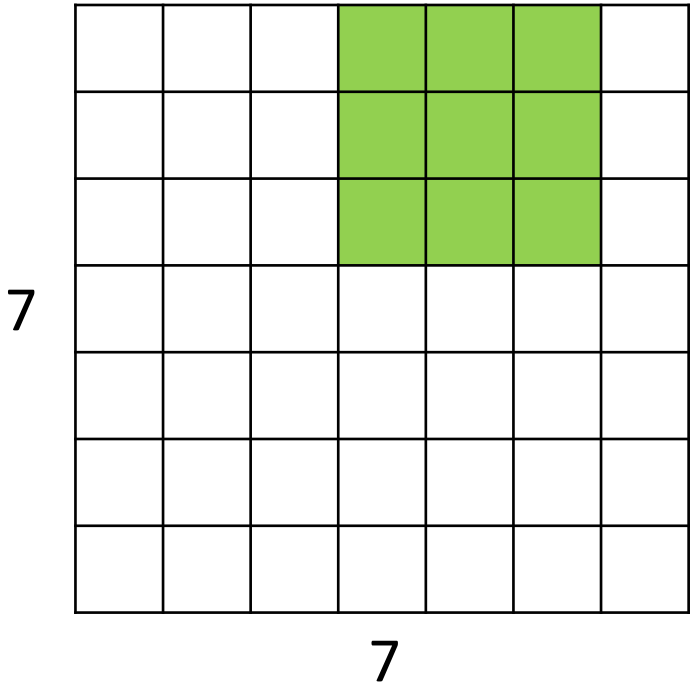
Convolution: Spatial Dimensions



Input: 7x7

Filter: 3x3

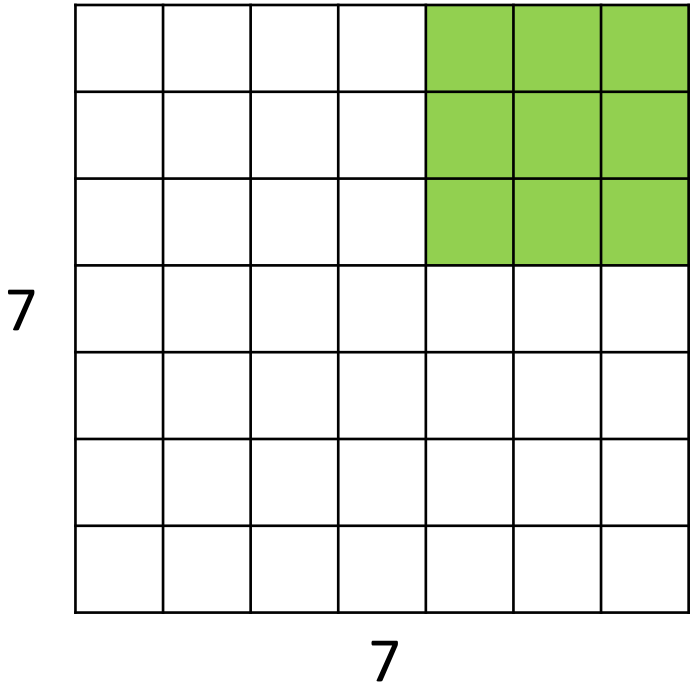
Convolution: Spatial Dimensions



Input: 7x7

Filter: 3x3

Convolution: Spatial Dimensions

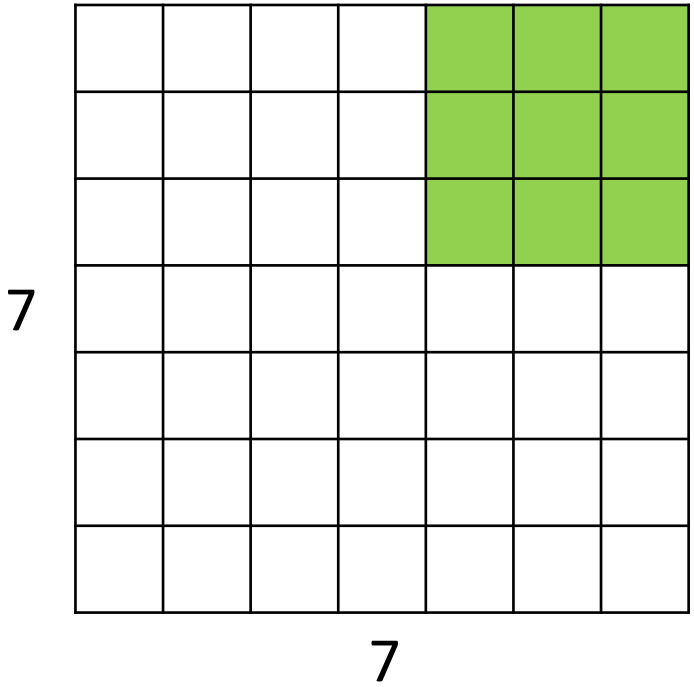


Input: 7x7

Filter: 3x3

Output: 5x5

Convolution: Spatial Dimensions



Input: 7x7

Filter: 3x3

Output: 5x5

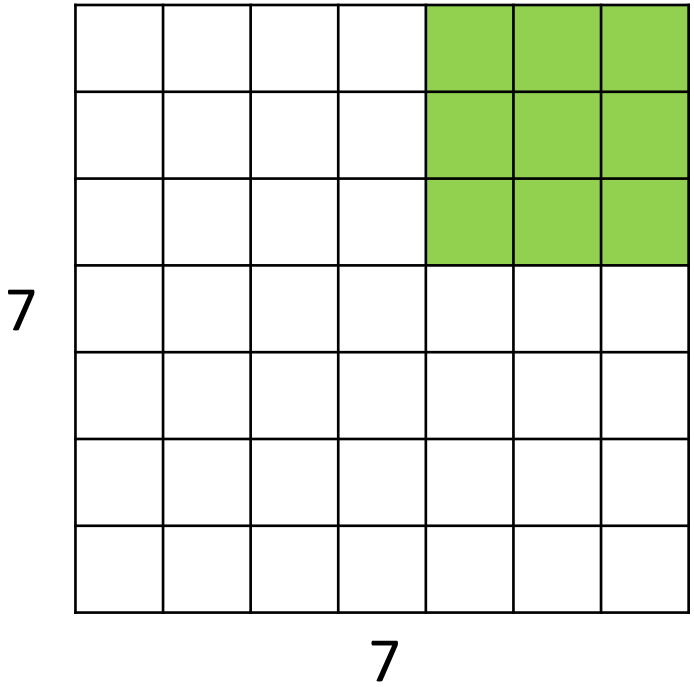
In general Input:

W

Filter: K

Output: W – K + 1

Convolution: Spatial Dimensions



Input: 7x7
Filter: 3x3
Output: 5x5

Problem: Feature maps shrink with each layer!

In general Input:
W
Filter: K
Output: W – K + 1

Convolution: Spatial Dimensions

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input: 7x7
Filter: 3x3
Output: 5x5

Problem: Feature maps shrink with each layer!

Solution: Add **padding** around the input before sliding the filter

In general
Input: W
Filter: K
Padding: P
Output: W – K + 1 + 2P

Convolution: Spatial Dimensions

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

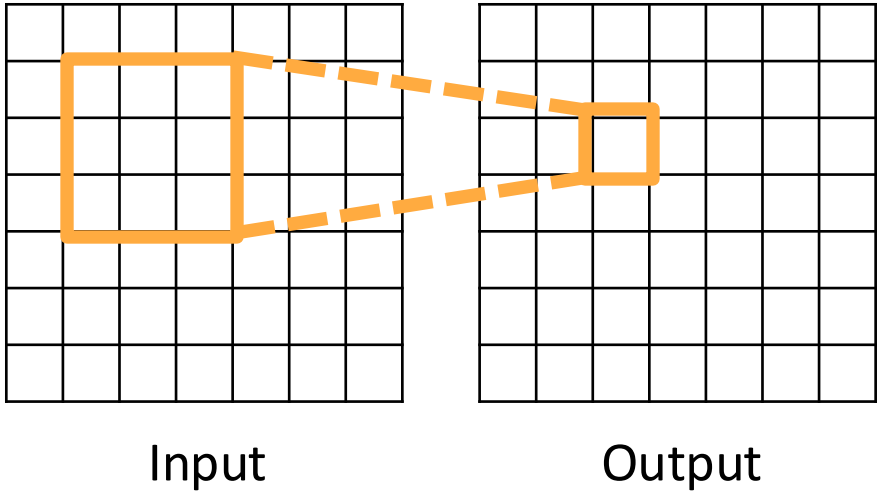
Input: 7x7
Filter: 3x3
Output: 5x5

In general
Input: W
Filter: K
Padding: P
Output: W – K + 1 + 2P

Common setting:
P = (K – 1) / 2
Means output has same size as input

Receptive Fields

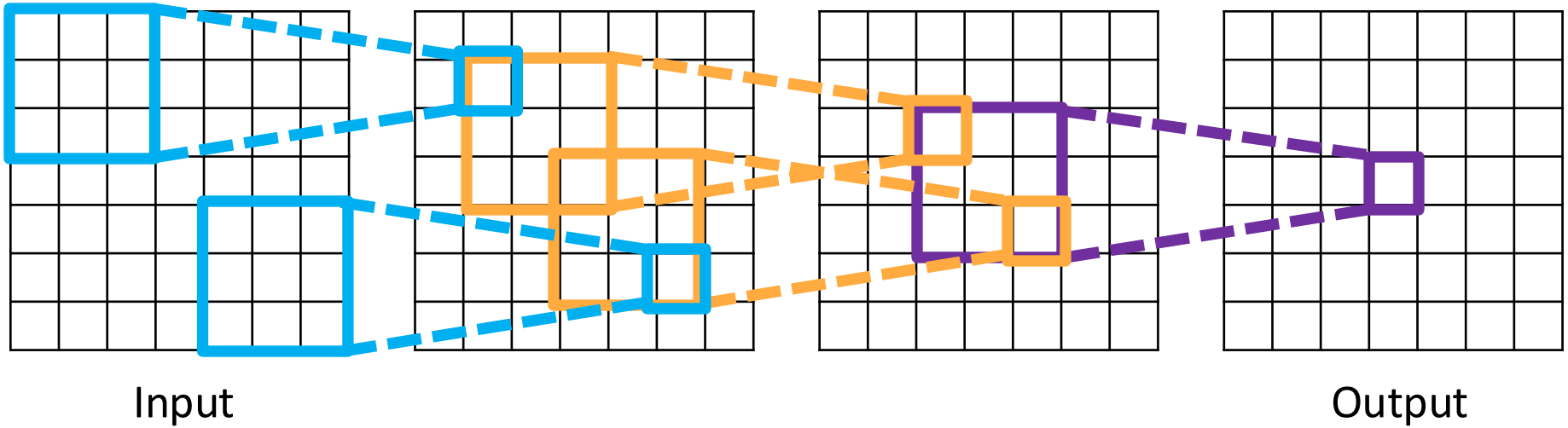
For convolution with **kernel size K**, each element in the output depends on a K x K receptive field in the input



Slide inspiration: Justin Johnson

Receptive Fields

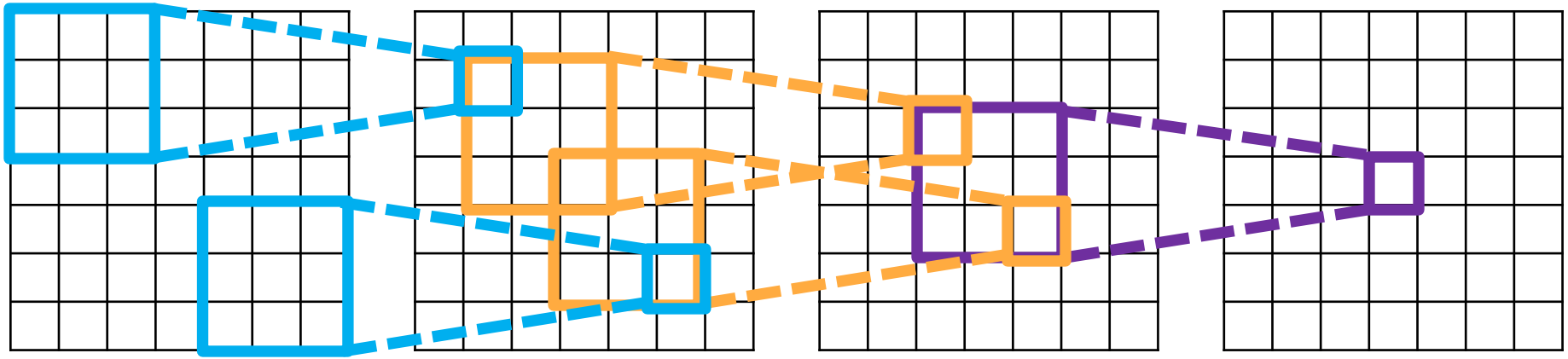
Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



Be careful – “receptive field in the input” vs. “receptive field in the previous layer”

Receptive Fields

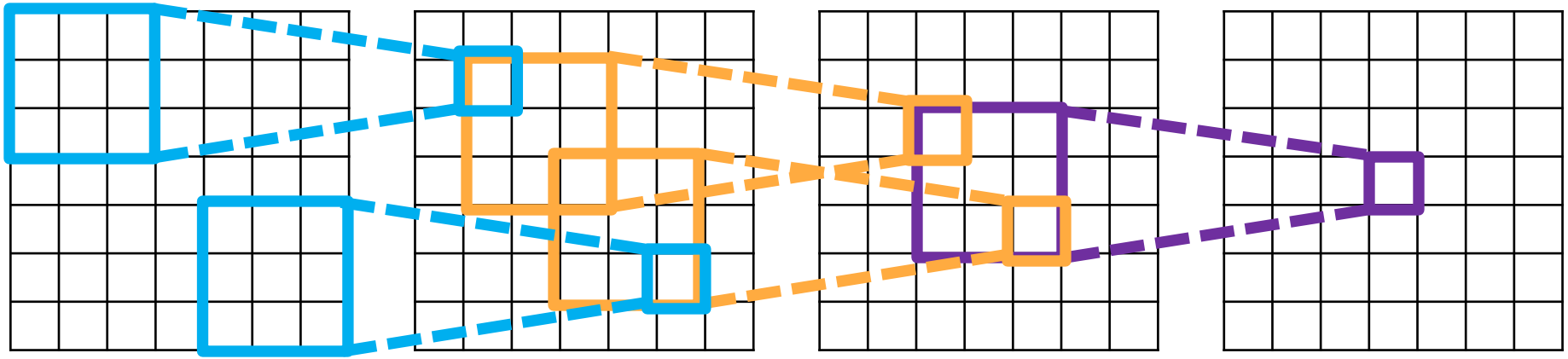
Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



Problem: For large images we need many layers for each output to “see” the whole image image

Receptive Fields

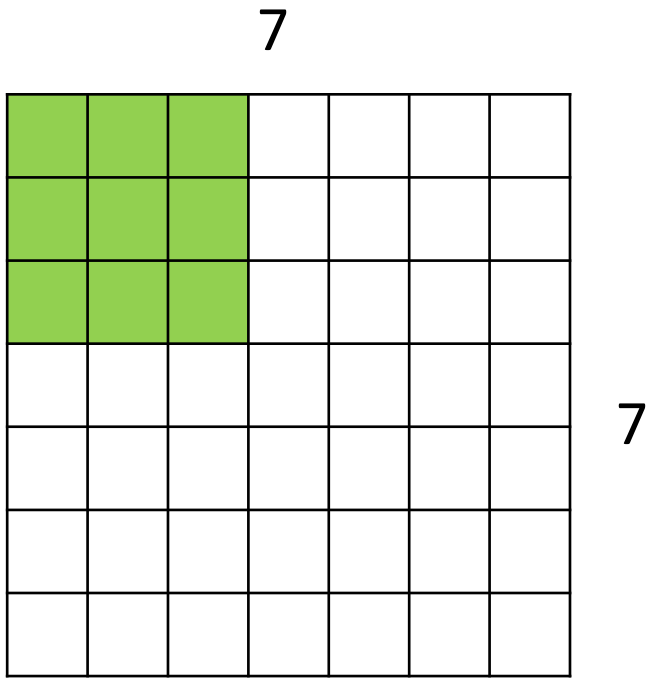
Each successive convolution adds K – 1 to the receptive field size
With L layers the receptive field size is 1 + L * (K – 1)



Problem: For large images we need many layers for each output to “see” the whole image image

Solution: Downsample inside the network

Strided Convolution

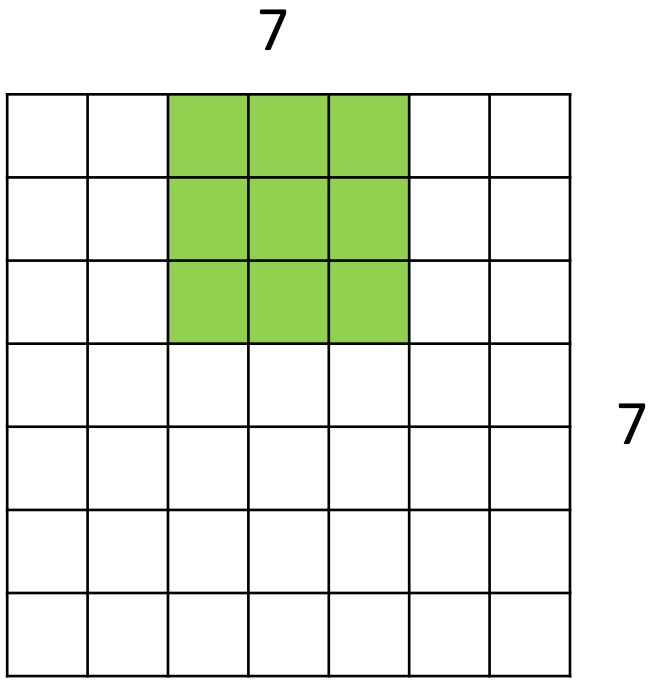


Input: 7x7

Filter: 3x3

Stride: 2

Strided Convolution

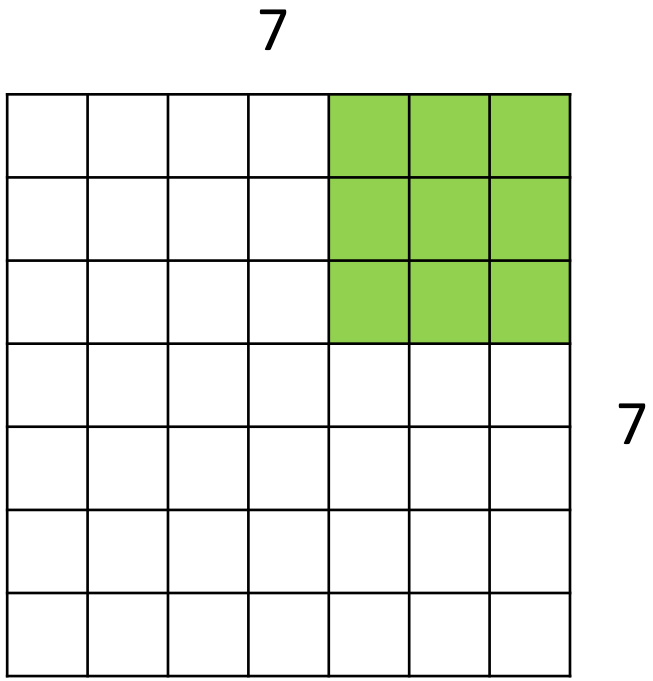


Input: 7x7

Filter: 3x3

Stride: 2

Strided Convolution



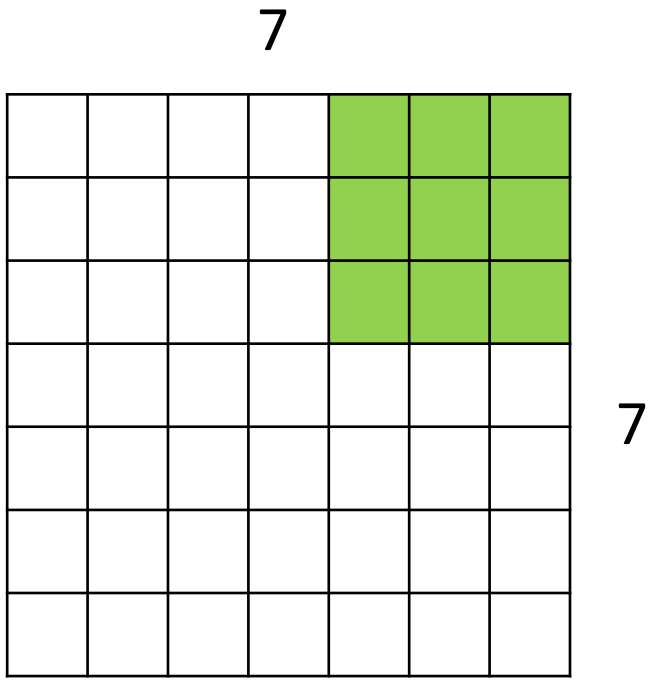
Input: 7x7

Filter: 3x3

Stride: 2

Output: 3x3

Strided Convolution



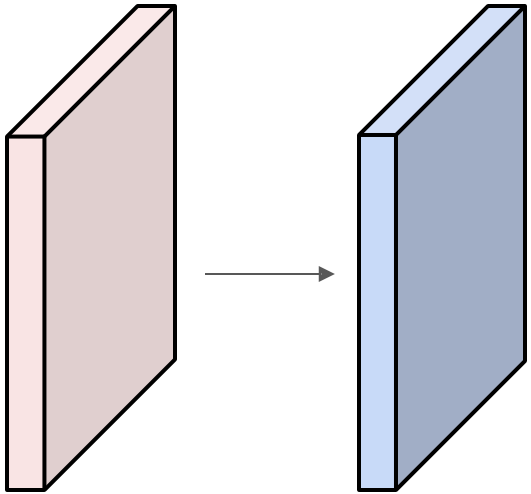
Input: 7x7
Filter: 3x3
Stride: 2
Output: 3x3

In general:
Input: W
Filter: K
Padding: P
Stride: S

→ Output:
$(W - K + 2P) / S + 1$

Convolution Example

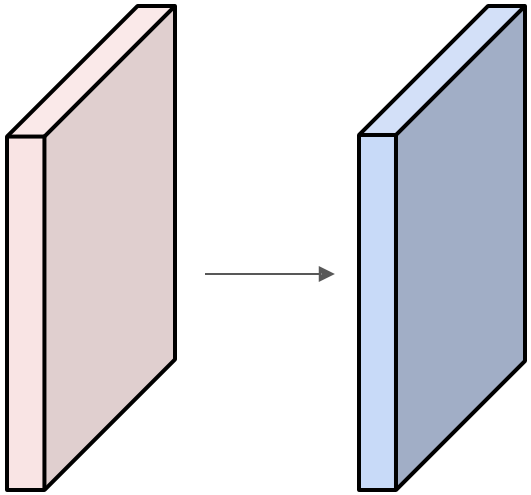
- Input volume: 3 x 32 x 32
- 10 5x5 filters with stride 1, pad 2
- Output volume size: ?



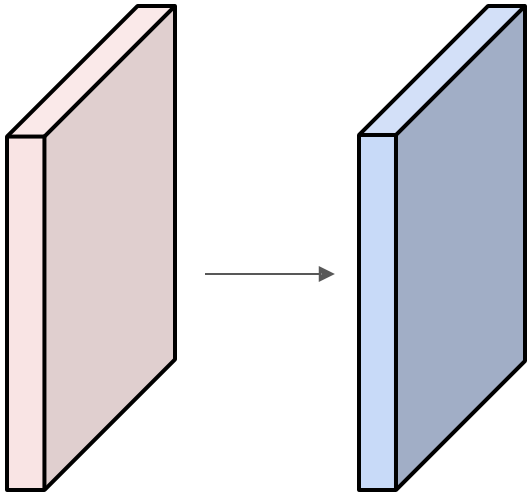
Convolution Example

- Input volume: 3 x 32 x 32
- 10 5x5 filters with stride 1, pad 2

- Output volume size: 10 x 32 x 32
- 32 = (32+2*2-5)/1+1

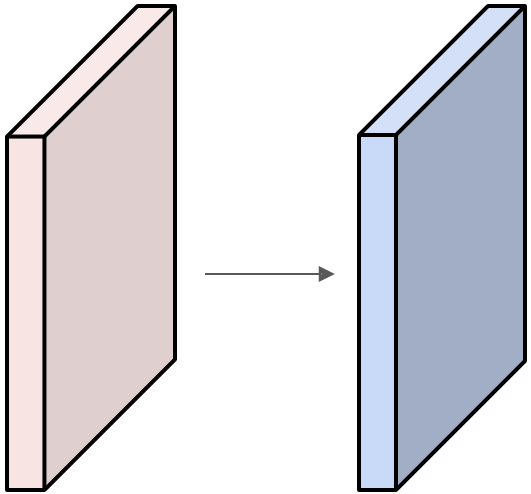


Convolution Example



- Input volume: 3 x 32 x 32
- 10 5x5 filters with stride 1, pad 2
- Output volume size: 10 x 32 x 32
- Number of learnable parameters: ?

Convolution Example



Input volume: 3 x 32 x 32

10 5x5 filters with stride 1, pad 2

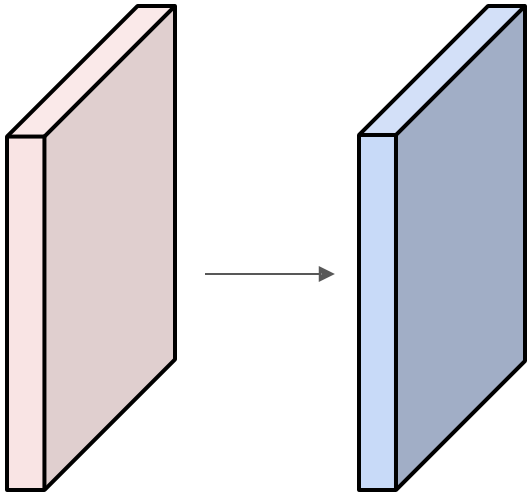
Output volume size: 10 x 32 x 32

Number of learnable parameters: 760

Parameters per filter: 3*5*5 + 1 (for bias) = 76

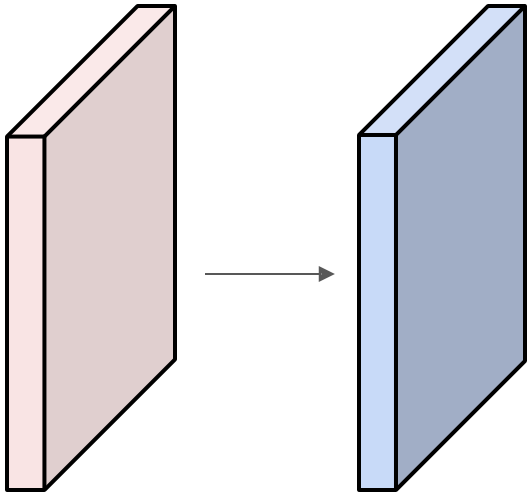
10 filters, so total is 10 * 76 = **760**

Convolution Example



- Input volume: 3 x 32 x 32
- 10 5x5 filters with stride 1, pad 2
- Output volume size: 10 x 32 x 32
- Number of learnable parameters: 760
- Number of multiply-add operations?

Convolution Example



Input volume: **3** x 32 x 32

10 **5x5** filters with stride 1, pad 2

Output volume size: **10 x 32 x 32**

Number of learnable parameters: 760

Number of multiply-add operations: **768,000x2**

10*32*32 = 10,240 outputs

Each output is the inner product of two **3**x**5x5** tensors (75 elems)

Total = 75*10240 = **768K**

PyTorch Convolution Layer

CLASS torch.nn.Conv2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=1*, *groups=1*, *bias=True*, *padding_mode='zeros'*, *device=None*, *dtype=None*) [SOURCE]

Applies a 2D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ can be precisely described as:

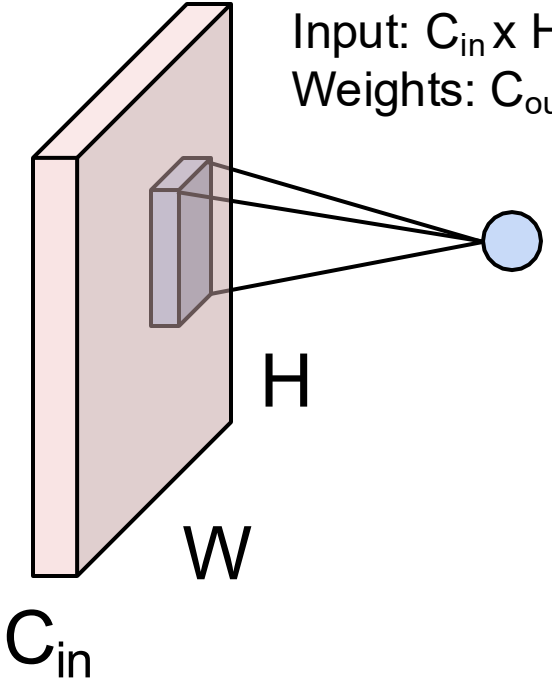
$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

We didn’t talk about groups or dilation…

Other Types of Convolution

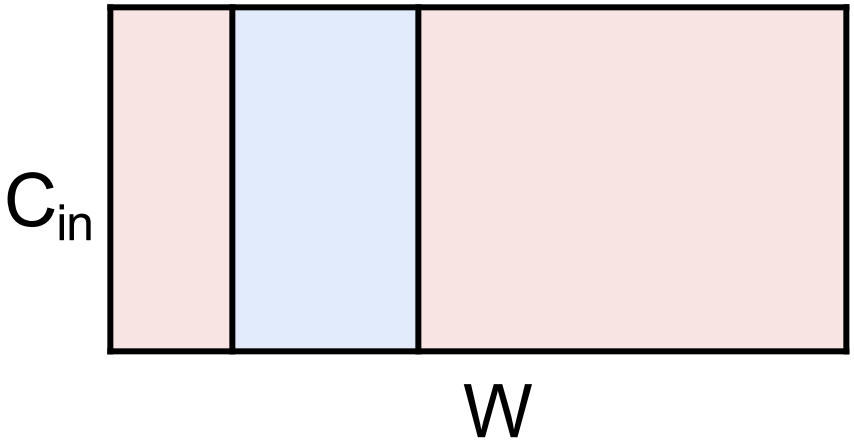
So far: 2D Convolution

Input: C_{in} x H x W
Weights: C_{out} x C_{in} x K x K



1D Convolution

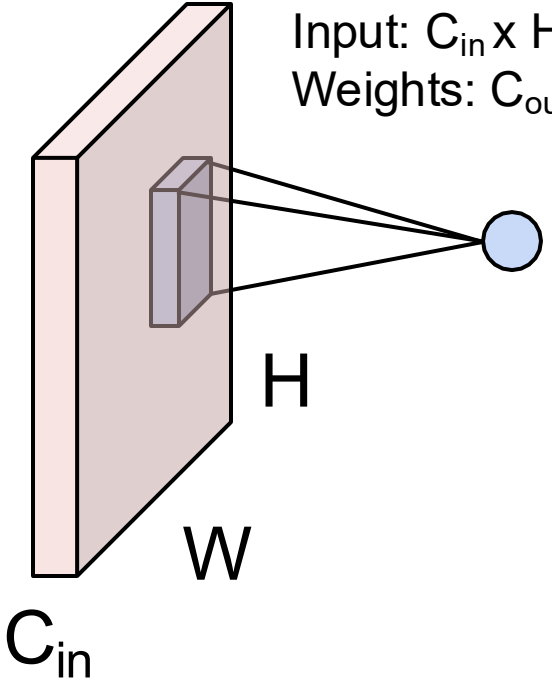
Input: C_{in} x W
Weights: C_{out} x C_{in} x K



Other Types of Convolution

So far: 2D Convolution

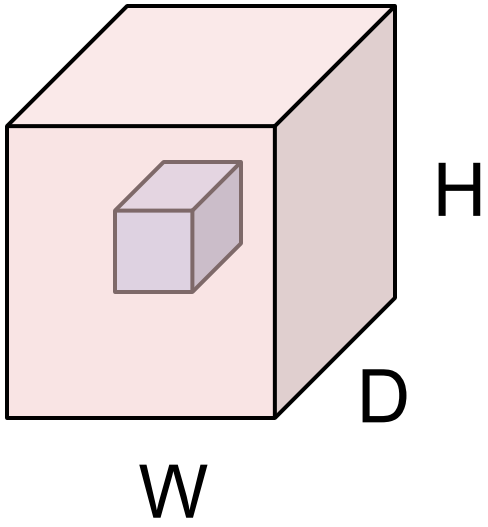
Input: C_{in} x H x W
Weights: C_{out} x C_{in} x K x K



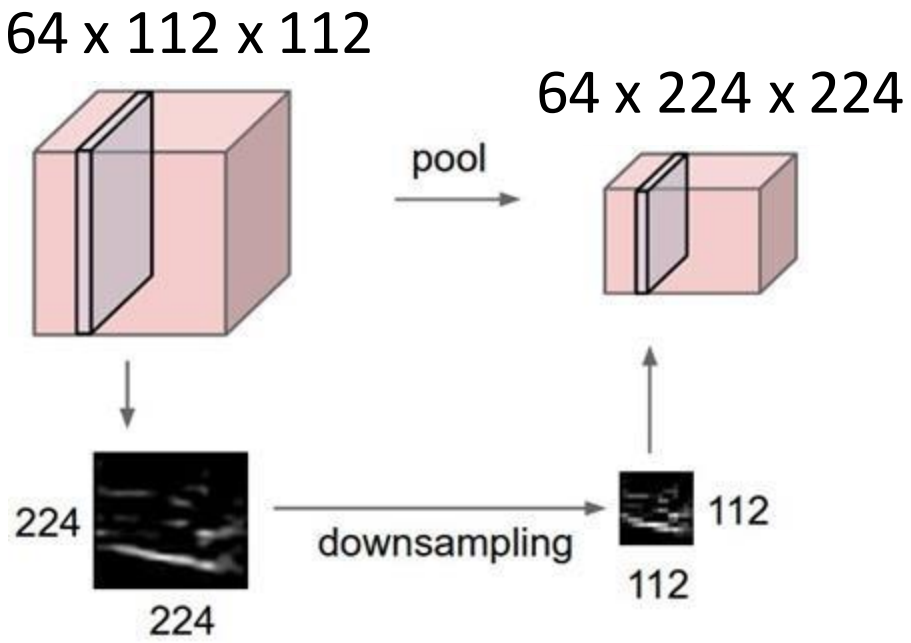
3D Convolution

Input: C_{in} x H x W x D
Weights: C_{out} x C_{in} x K x K x K

C_{in}-dim vector at each point in the volume



Pooling Layers: Another way to downsample



Given an input C x H x W, downsample each 1 x H x W plane

Hyperparameters:
Kernel Size
Stride
Pooling function

Pooling Layers: Another way to downsample

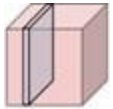
Single depth slice

x

1	1	2	4
5	**6**	7	**8**
3	2	1	0
1	2	3	**4**

y

64 x 224 x 224

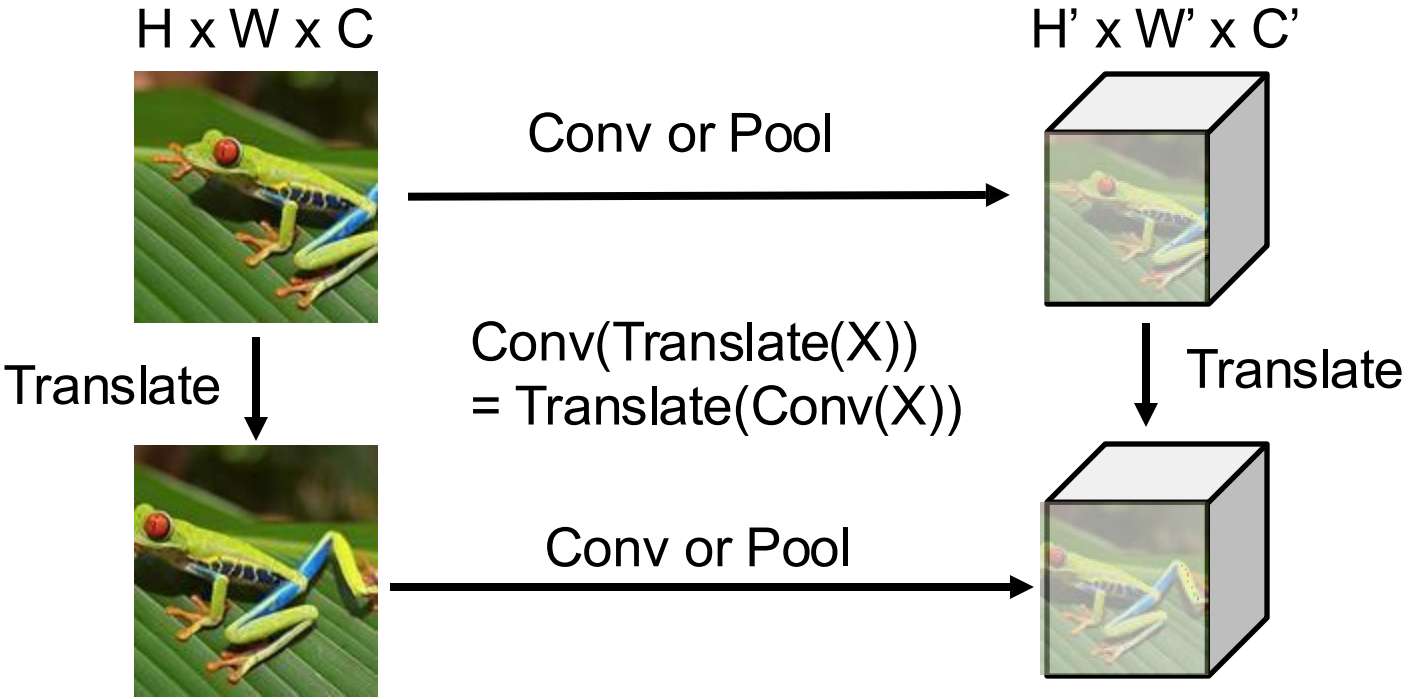


Max pooling with 2x2 kernel size and stride 2

6	8
3	4

Gives **invariance** to small spatial shifts. No learnable parameters.

Convolution and Pooling: Translation Equivariance

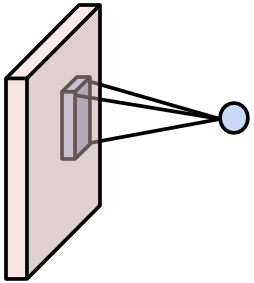


Intuition: Features of images don't depend on their location in the image

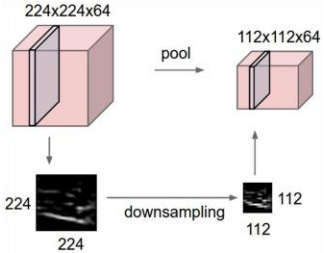
Training CNNs and CNN Architectures

Components of CNNs

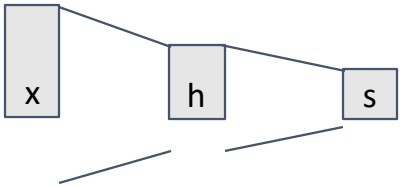
Convolution Layers



Pooling Layers



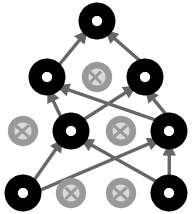
Fully-Connected Layers



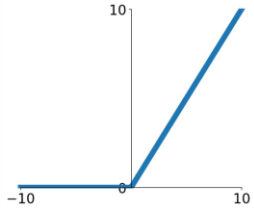
Normalization Layers

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Dropout (sometimes)



Activation Functions



LayerNorm

High-level Idea: Learn parameters that let us scale / shift the input data

1. Normalize input data
2. Scale / shift using learned parameters

$$\mathbf{x}: \; N \times D$$

Normalize

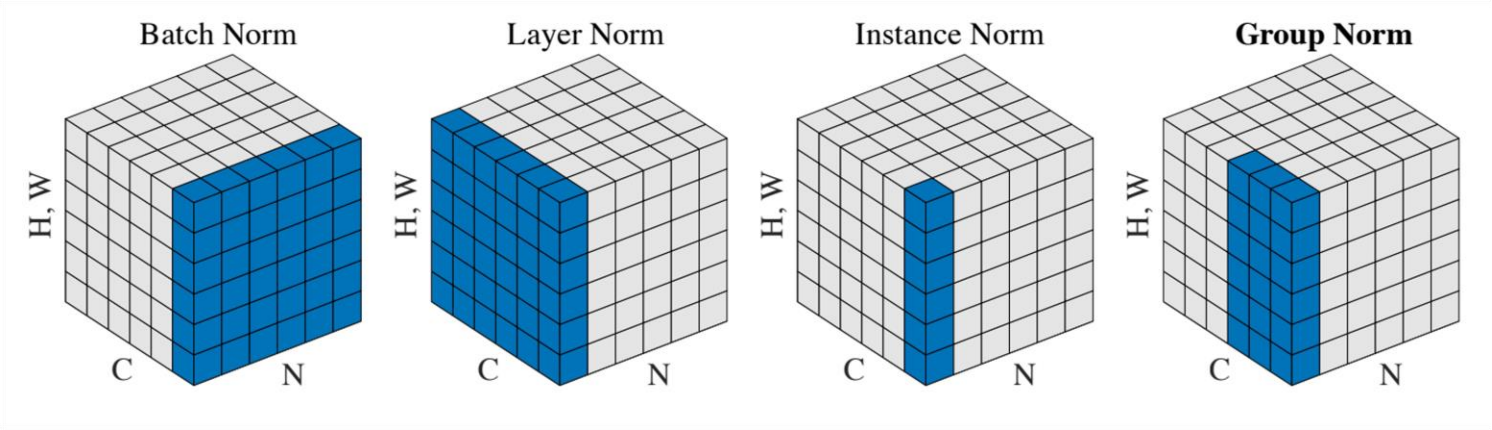
Statistics calculated per batch ➔ $\boldsymbol{\mu}, \boldsymbol{\sigma}: \; N \times 1$

Learned parameters applied to each sample ➔ $\gamma, \beta: \; 1 \times D$

$$\mathbf{y} = \gamma(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \beta$$

Ba, Kiros, and Hinton, “Layer Normalization”, arXiv 2016

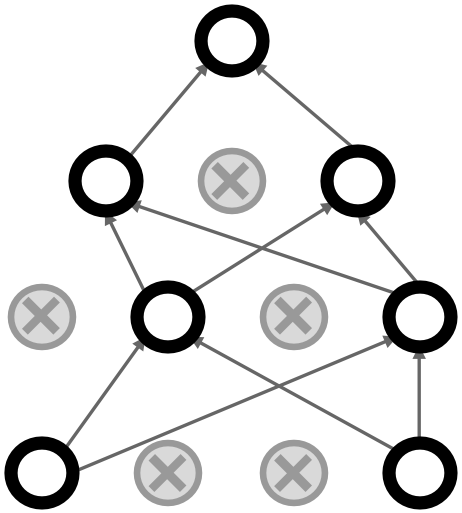
Other Normalization Layers



Wu and He, “Group Normalization”, ECCV 2018

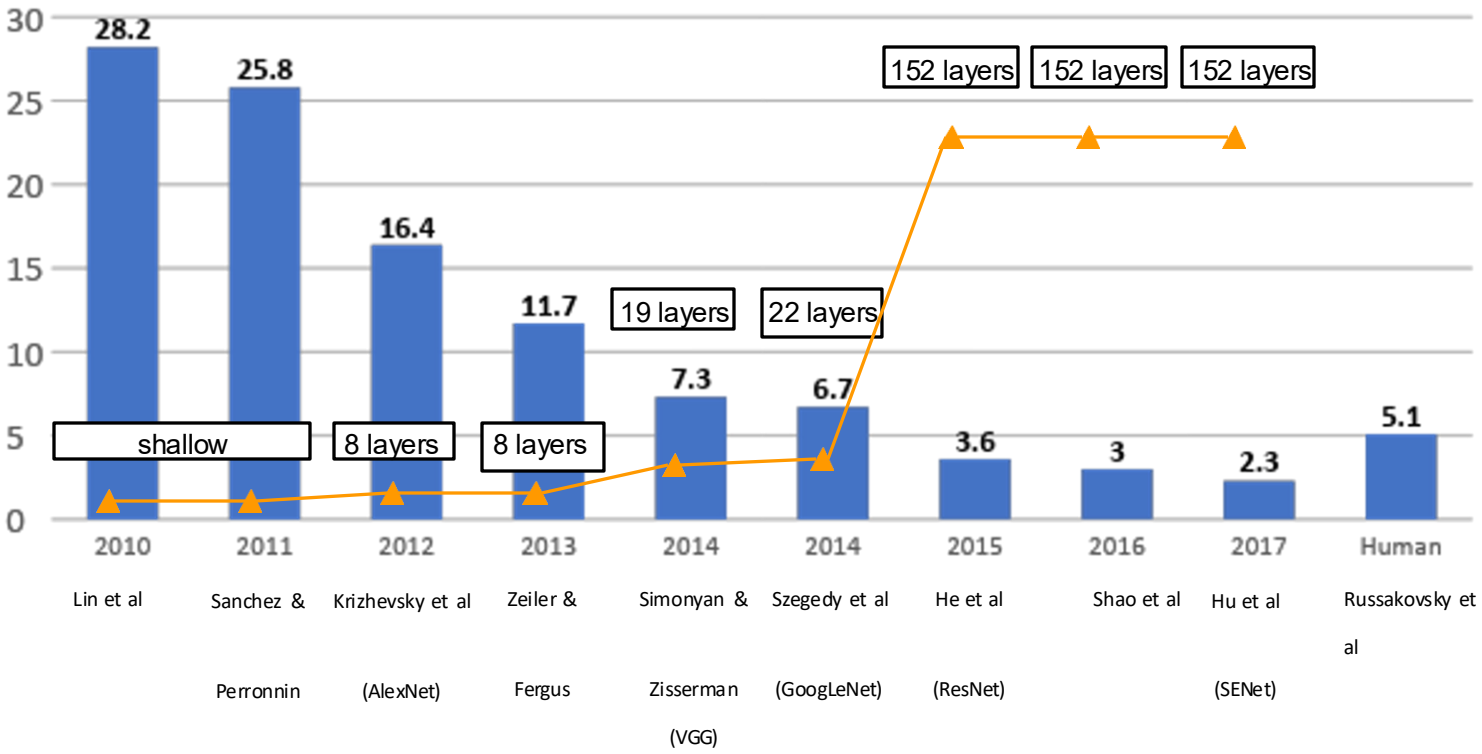
Regularization: Dropout

How can this possibly be a good idea?



Forces the network to have a redundant representation;
Prevents co-adaptation of features

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



Case Study: VGGNet

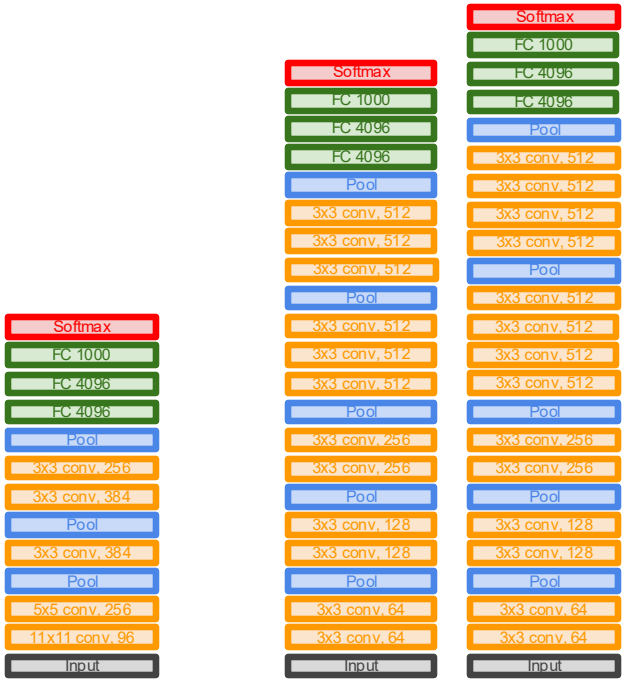
[Simonyan and Zisserman, 2014]

Small filters, Deeper networks

8 layers (AlexNet)
-> 16 - 19 layers (VGG16Net)

Only 3x3 CONV stride 1, pad 1
and 2x2 MAX POOL stride 2

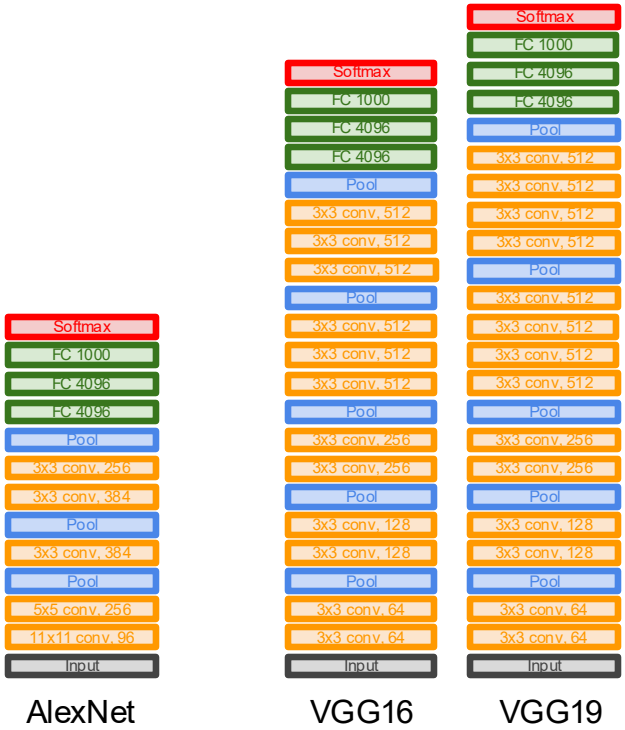
11.7% top 5 error in ILSVRC'13
(ZFNet)
-> 7.3% top 5 error in ILSVRC'14



Case Study: VGGNet

[Simonyan and Zisserman, 2014]

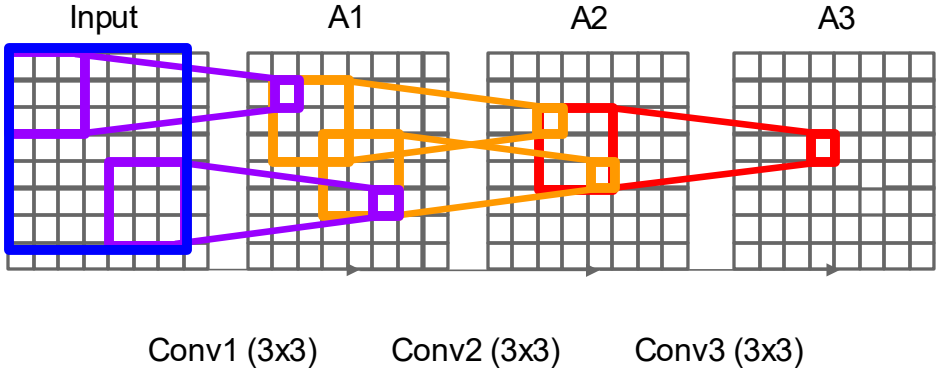
Q: Why use smaller filters? (3x3 conv)



Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



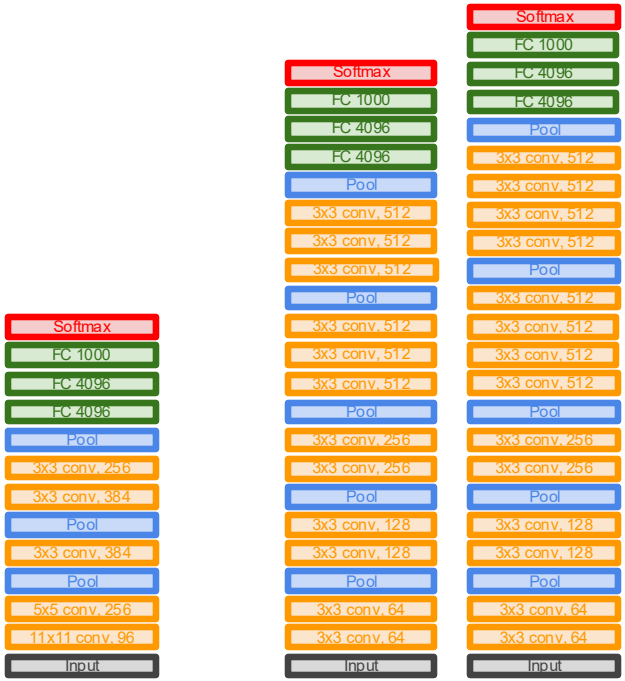
Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Q: Why use smaller filters? (3x3 conv)

Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer

[7x7]



AlexNet VGG16 VGG19

Case Study: VGGNet

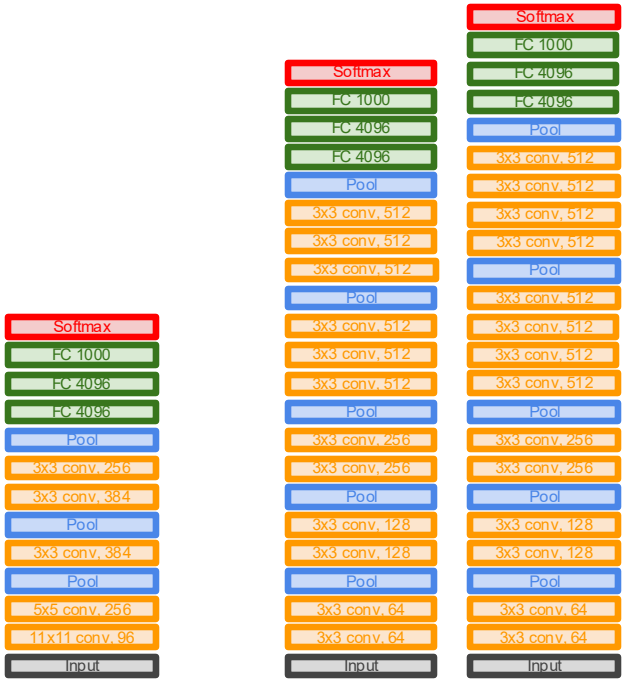
[Simonyan and Zisserman, 2014]

Q: Why use smaller filters? (3x3 conv)

Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer

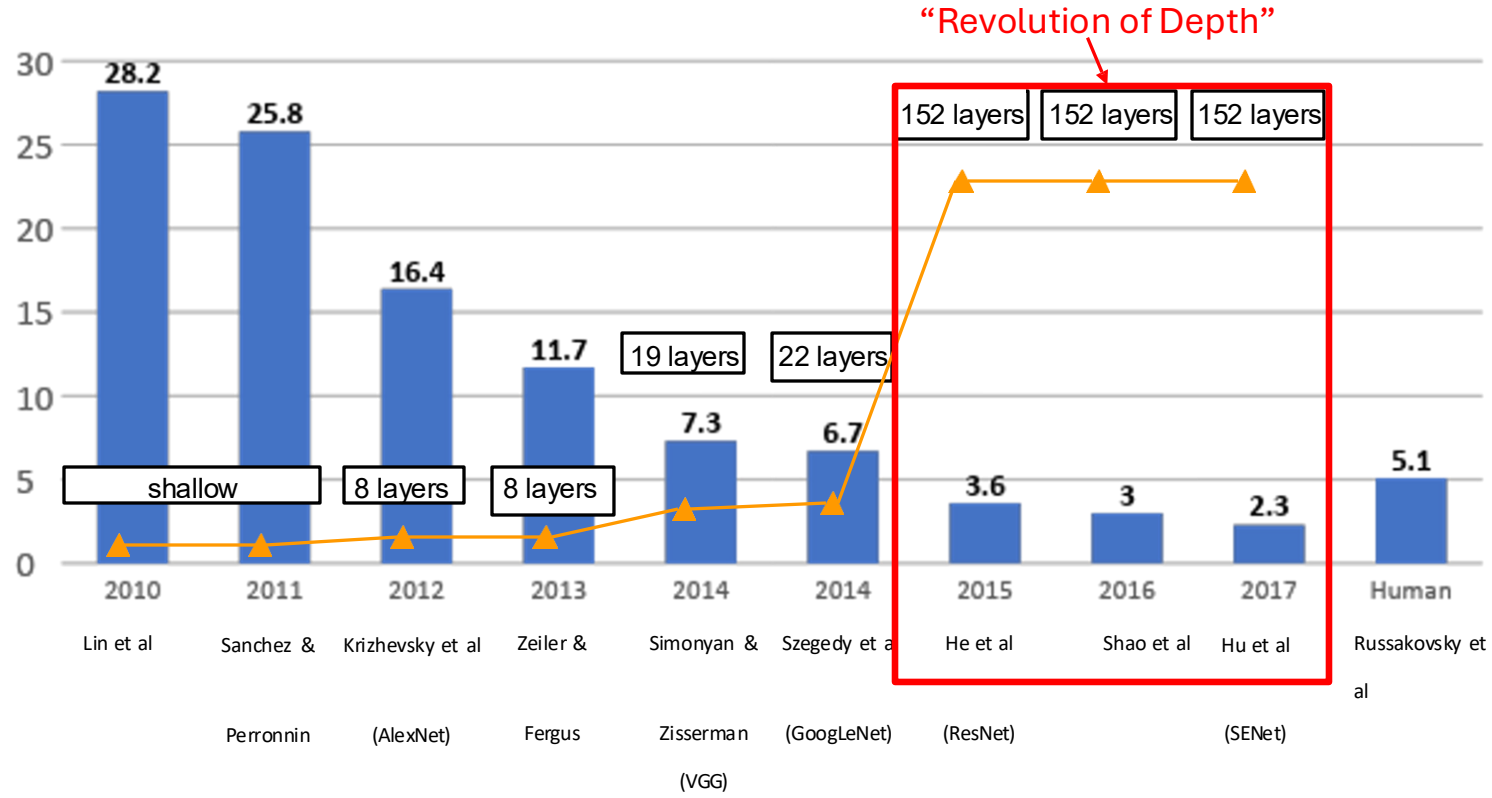
But deeper, more non-linearities

And fewer parameters: $3 * (3^2C^2)$ vs. 7^2C^2 for C channels per layer



AlexNet VGG16 VGG19

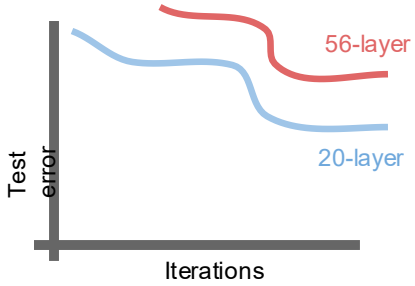
ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

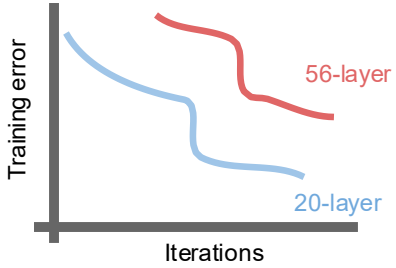


Case Study: ResNet

[He et al., 2015]

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?





56-layer model performs worse on both test and training error
-> The deeper model performs worse, but it’s not caused by overfitting!

Case Study: ResNet

[He et al., 2015]

Fact: Deep models have more representation power (more parameters) than shallower models.

Hypothesis: the problem is an *optimization* problem,
deeper models are harder to optimize

Case Study: ResNet

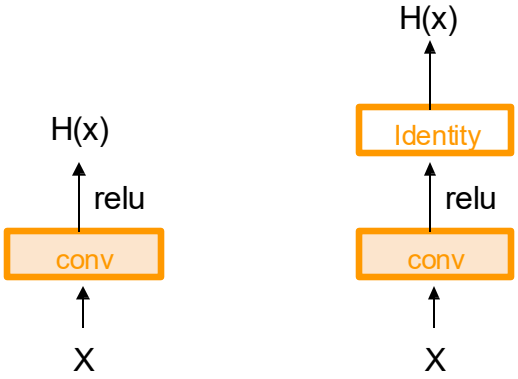
[He et al., 2015]

Fact: Deep models have more representation power (more parameters) than shallower models.

Hypothesis: the problem is an *optimization* problem, deeper models are harder to optimize

What should the deeper model learn to be at least as good as the shallower model?

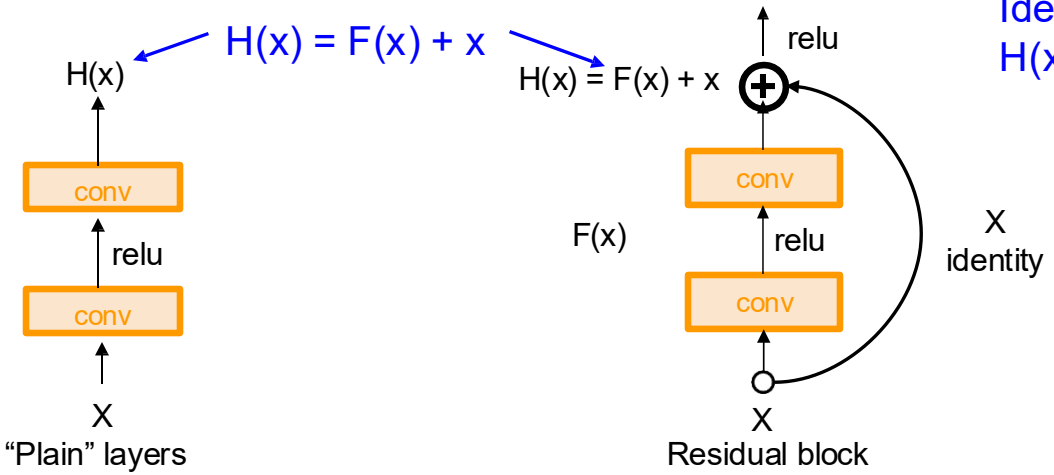
A solution by construction is copying the learned layers from the shallower model and setting additional layers to identity mapping.



Case Study: ResNet

[He et al., 2015]

Solution: Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping



Identity mapping:
H(x) = x if F(x) = 0

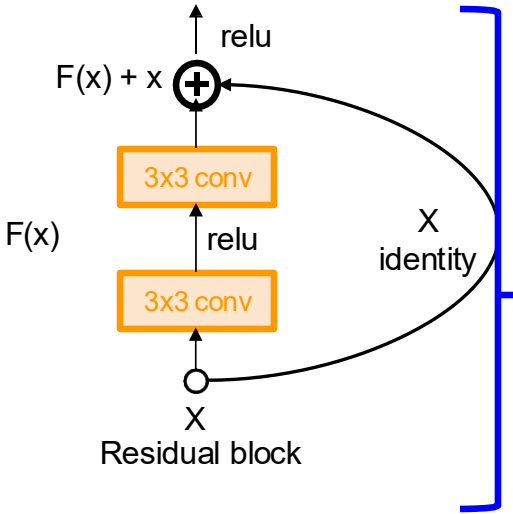
Use layers to fit **residual** F(x) = H(x) - x instead of H(x) directly

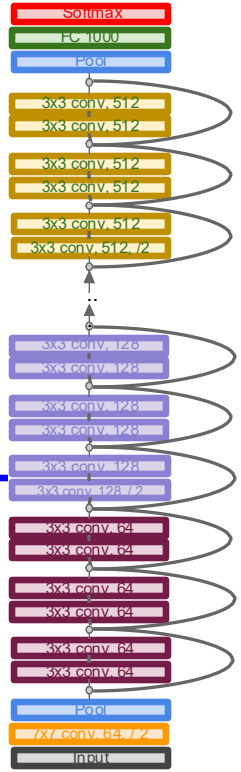
Case Study: ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers



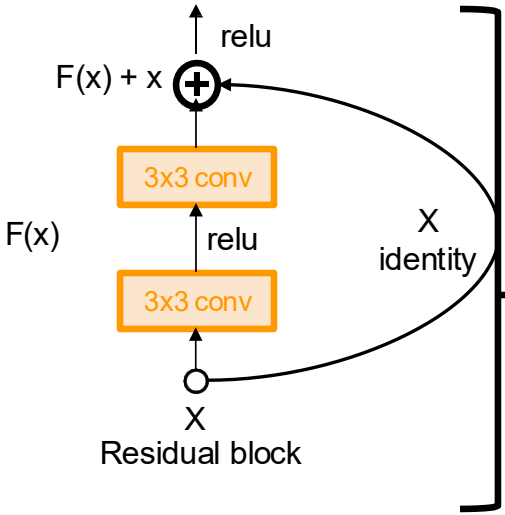


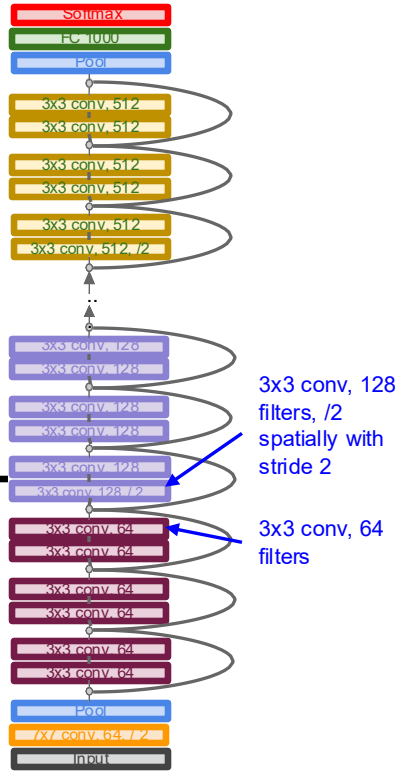
Case Study: ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension) Reduce the activation volume by half.



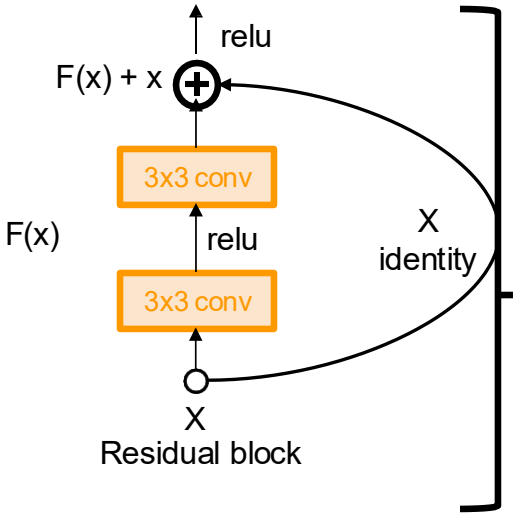


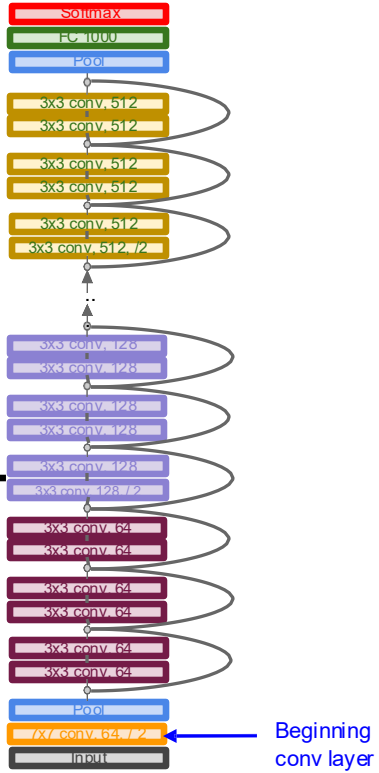
Case Study: ResNet

[He et al., 2015]

Full ResNet architecture:

- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning (stem)

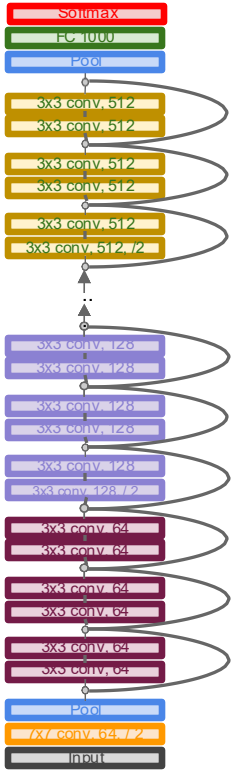




Case Study: ResNet

[He et al., 2015]

Total depths of 18, 34, 50, 101, or 152 layers for ImageNet

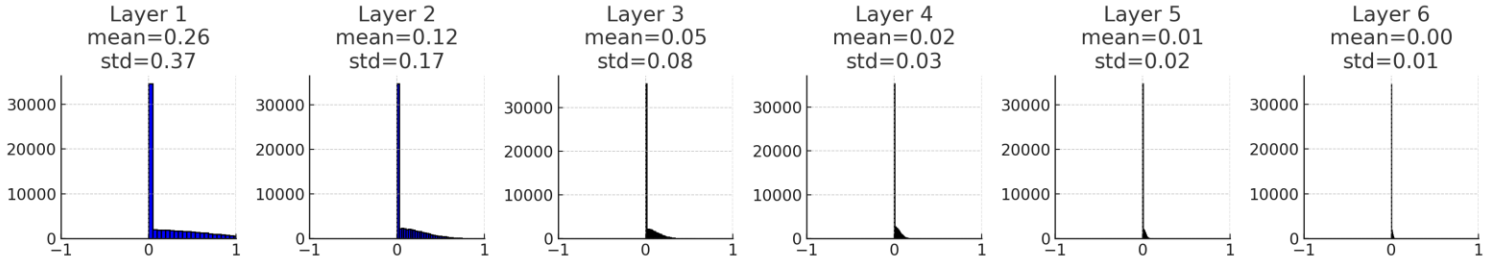


How to initialize weights in neural network layers?

Weight Initialization Case: Values too small

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)  # Small weight init
    x = np.maximum(0, x.dot(W))  # ReLU activation
    hs.append(x)
```

All activations tend to zero for deeper network layers

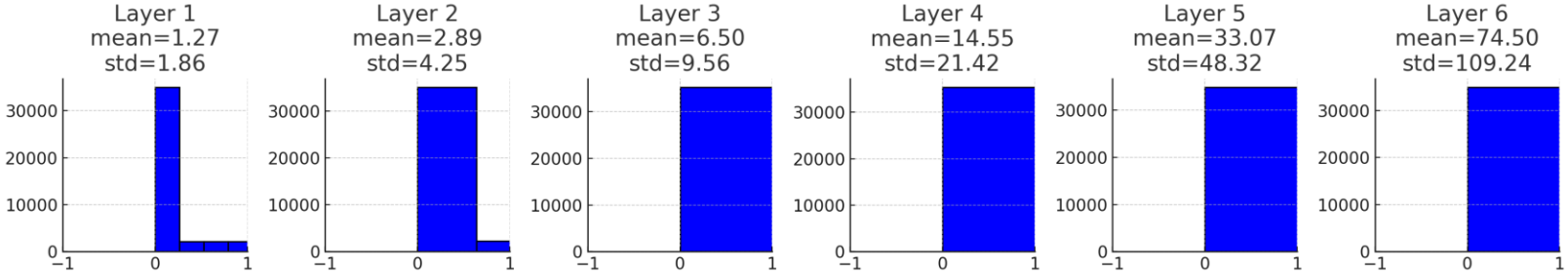


Weight Initialization Case: Values too large

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)  # Small weight init
    x = np.maximum(0, x.dot(W))  # ReLU activation
    hs.append(x)
```

Activations blow up quickly

Increase std of initial weights from 0.01 to 0.05

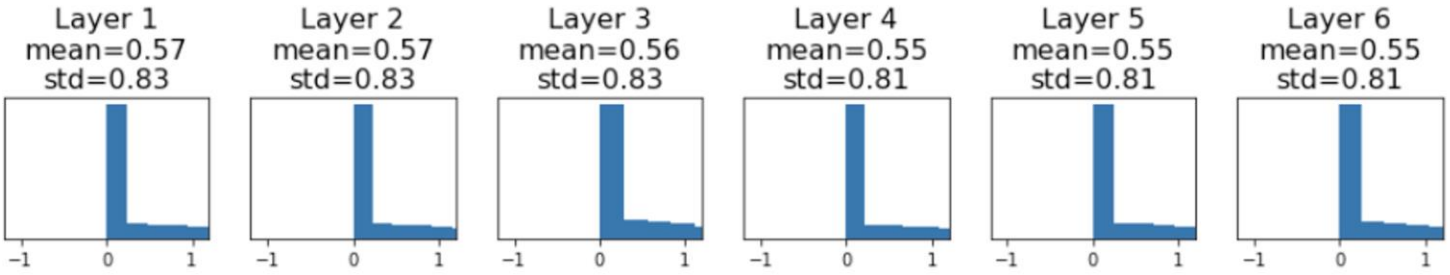


One solution: Kaiming Initialization

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

ReLU correction: std = sqrt(2 / Din)

"Just right": Activations are nicely scaled for all layers!



He et al, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification", ICCV 2015

How to train CNNs?

Data Preprocessing
Data augmentation
Transfer Learning
Hyperparameter Selection

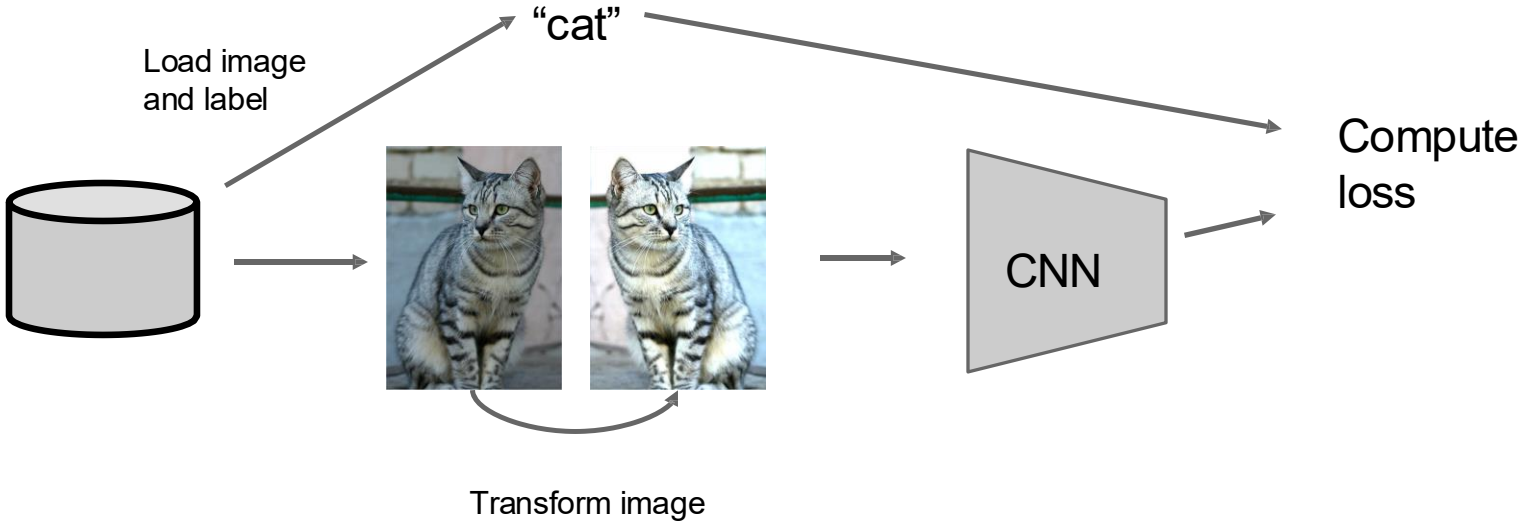
Image Normalization:

center and scale for each channel

- Subtract per-channel mean and
  Divide by per-channel std (almost all modern models)
  (stats along each channel = 3 numbers)
- Requires pre-computing means and std for each pixel channel (given your dataset)

```
norm_pixel[i,j,c] = (pixel[i,j,c] - np.mean(pixel[:,:,c])) / np.std(pixel[:,:,c])
```

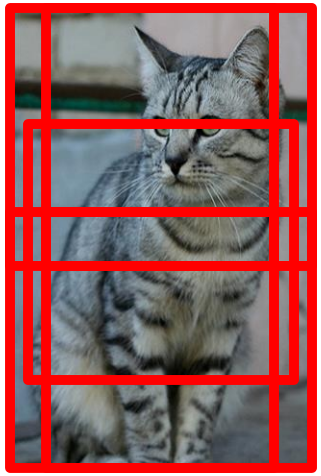
Regularization: Data Augmentation



Data Augmentation
Horizontal Flips

Data Augmentation

Random crops and scales



Training: sample random crops / scales

ResNet:

1. Pick random L in range [256, 480]
2. Resize training image, short side = L
3. Sample random 224 x 224 patch

Test Time Augmentation: average a fixed set of crops

ResNet:

1. Resize image at 5 scales: {224, 256, 384, 480, 640}
2. For each size, use 10 224 x 224 crops: 4 corners + center, + flips

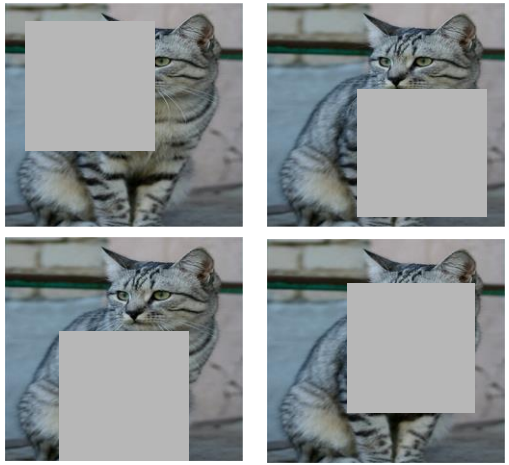
Data Augmentation

Color Jitter

Simple: Randomize contrast and brightness

Regularization: Cutout

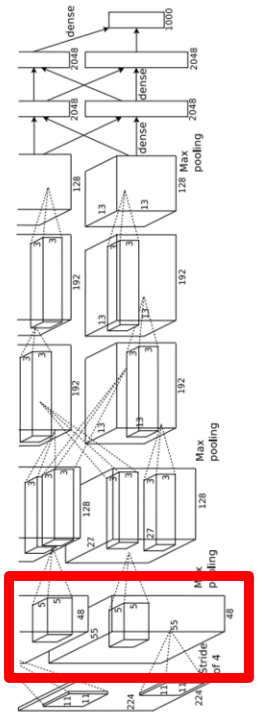
Training: Set random image regions to zero
Testing: Use full image

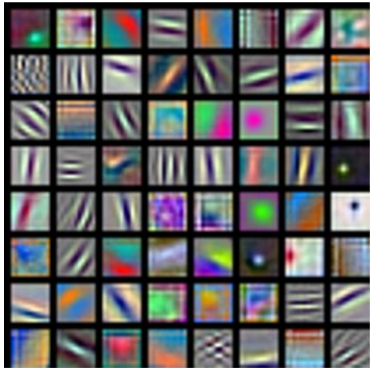


Works very well for small datasets like CIFAR, less common for large datasets like ImageNet

DeVries and Taylor, “Improved Regularization of Convolutional Neural Networks with Cutout”, arXiv 2017

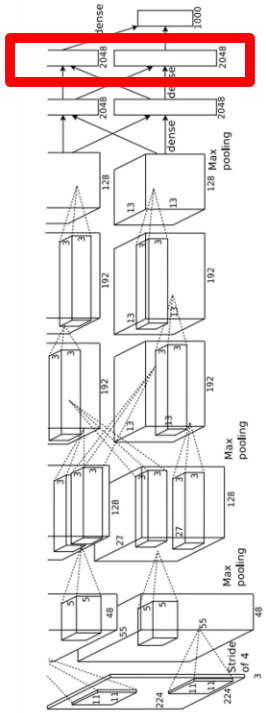
Transfer Learning with CNNs





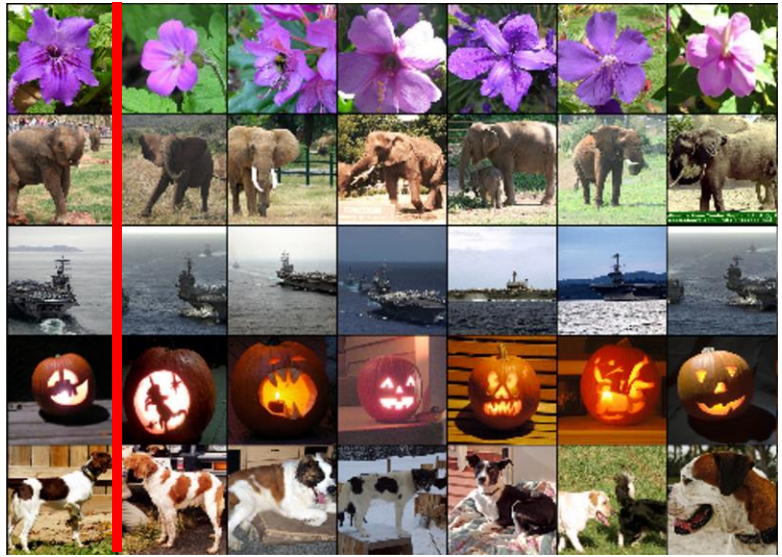
AlexNet:
64 x 3 x 11 x 11

Transfer Learning with CNNs



Test image

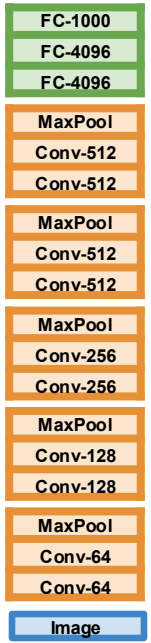
L2 Nearest neighbors in <u>feature</u> space



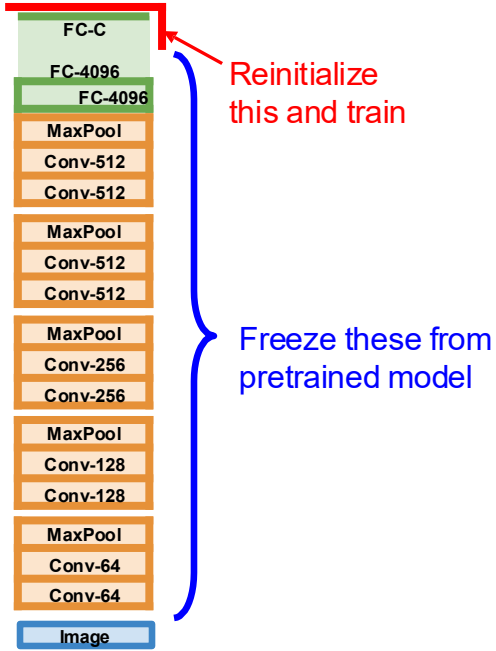
Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

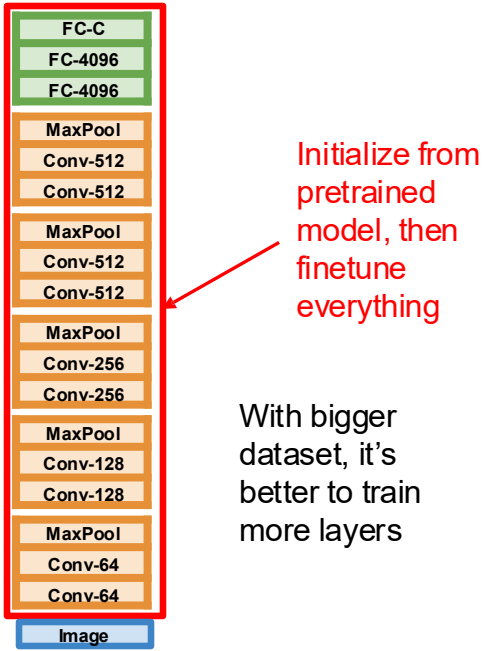
1. Train on Imagenet



2. Small Dataset (C classes)



3. Bigger dataset



With bigger dataset, it's better to train more layers

Choosing Hyperparameters

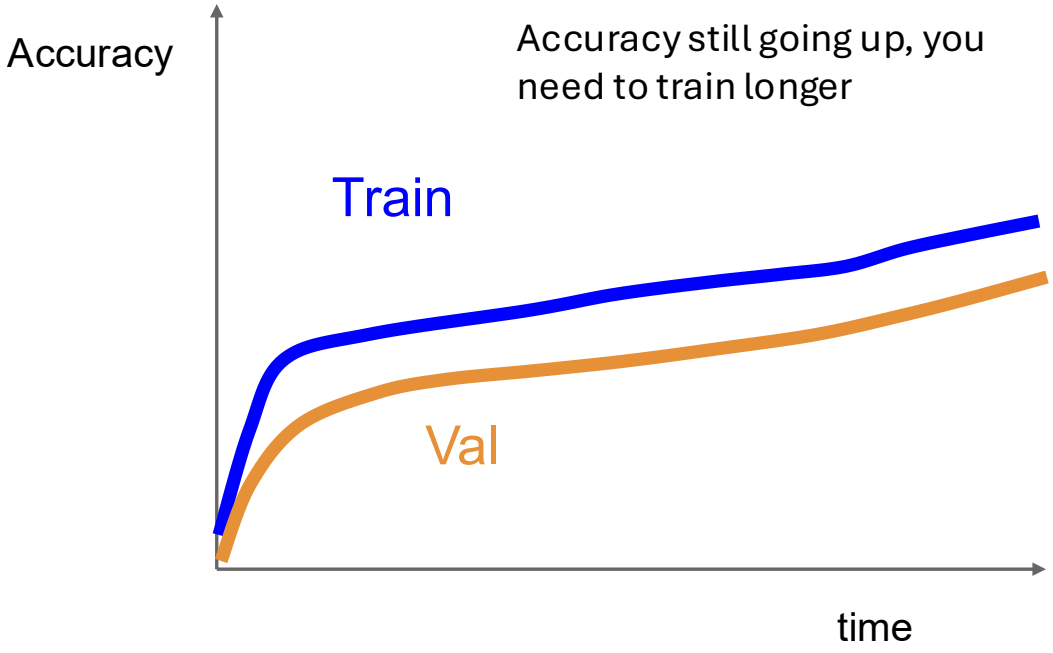
Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down

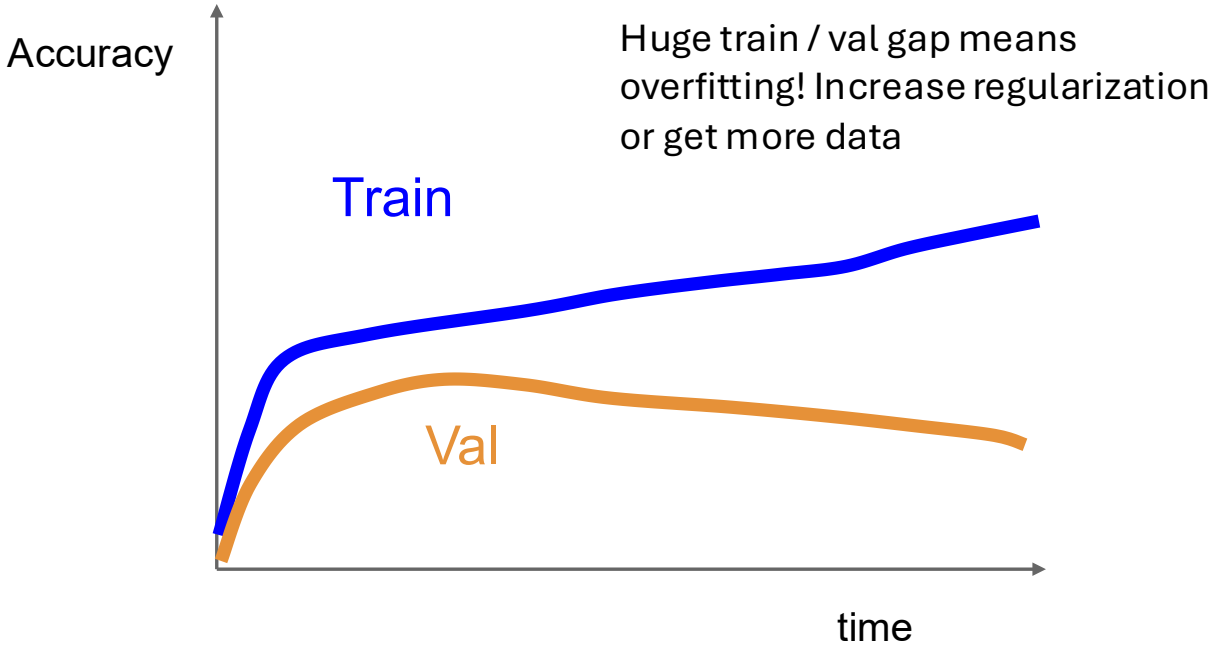
Use the architecture from the previous step, use all training data, turn on small weight decay, find a learning rate that makes the loss drop significantly within ~100 iterations

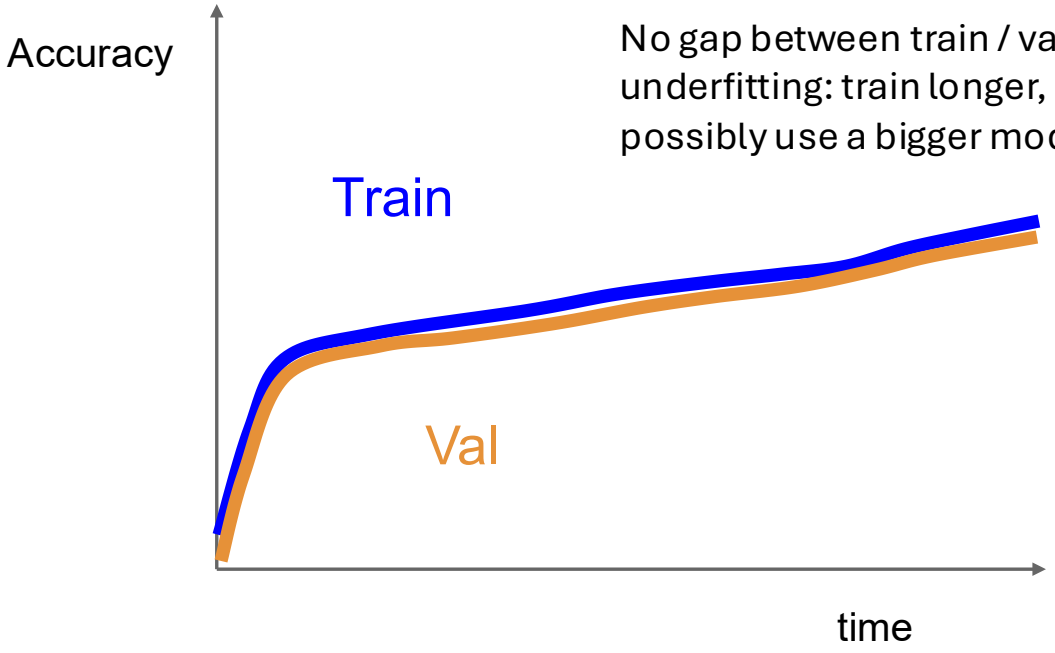
Good learning rates to try: 1e-1, 1e-2, 1e-3, 1e-4, 1e-5

Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down
Step 4: Coarse grid of hyperparams, train for ~1-5 epochs
Step 5: Refine grid, train longer
Step 6: Look at loss and accuracy curves (next slides)


Accuracy
Accuracy still going up, you need to train longer
Train
Val
time


Accuracy
Huge train / val gap means overfitting! Increase regularization or get more data
Train
Val
time



No gap between train / val means underfitting: train longer, can possibly use a bigger model

Choosing Hyperparameters

Step 1: Check initial loss
Step 2: Overfit a small sample
Step 3: Find LR that makes loss go down
Step 4: Coarse grid, train for ~1-5 epochs
Step 5: Refine grid, train longer
Step 6: Look at loss and accuracy curves
Step 7: GOTO step 5